

Corso di Laurea Magistrale in Ingegneria e Scienze Informatiche

Fancy Title

Tesi di laurea in:
SUPERVISOR'S COURSE NAME

Relatore

Prof. Supervisor Here

Candidato

Candidate Name

Correlatori

Dott. CoSupervisor 1

Dott. CoSupervisor 2

Abstract

Max 2000 characters, strict.

Optional. Max a few lines.

Contents

Abstract	iii
1 Introduction	1
2 Descrizione del problema	3
3 stato dell'arte	9
4 Formulazione matematica	15
4.1 Procedure di riduzione	18
4.1.1 Modifica delle capacità degli sprint	19
4.1.2 Modifica dei pesi delle user story	19
5 Euristiche per la pianificazione degli sprint	21
5.1 Euristiche greedy e di scambio del lavoro precedente	21
5.1.1 GreedyHeuristic: costruzione sprint-per-sprint	21
5.1.2 QuickGreedyHeuristic: versione veloce basata su knapsack .	23
5.1.3 ExchangeHeuristic: miglioramento locale per scambi	26
5.2 A Lagrangian Heuristic	29
5.3 Aggiornamenti ai metodi di cutting plane	34
5.4 Funzioni obiettivo	36
5.4.1 Formulazioni matematiche	37
5.4.2 Obiettivi delle diverse funzioni	38
5.4.3 Ruolo dei parametri modificatori	39
5.4.4 Confronto sperimentale	39
6 Risultati ottenuti	41
6.1 dati utilizzati	41
6.1.1 Raggruppamento dei progetti	42
6.1.2 Generazione di nuove istanze di test	45
6.1.3 Benchmark suite generato	46
6.1.4 Caratteristiche strutturali delle istanze	47

CONTENTS

6.2	KPI di valutazione	49
6.2.1	KPI computazionali	49
6.2.2	KPI di qualità della pianificazione	50
6.2.3	Interpretazione congiunta dei KPI	52
6.3	risultati computazionali	53
6.4	variazione dei parametri	55
6.4.1	Riduzione sprint	55
6.4.2	Riduzione capacità degli sprint	57
6.4.3	metodo aggiornato	61
6.5	Confronto funzioni obbiettivo	62
6.5.1	funzuone obbiettovo (5.23)	64
6.5.2	Funzione obbiettivo (5.24) e (5.25)	65
7	Conclusioni	67
		69
	Bibliography	69

List of Figures

LIST OF FIGURES

List of Listings

5.1	GreedyHeuristic	22
5.2	QuickGreedyHeuristic	25
5.3	ExchangeHeuristic	28
5.4	Algorithm LagrangianHeuristic	33
5.5	Algoritmo multi-cut per la callback di generazione tagli	35
5.6	Generazione corretta dei tagli di precedenza OR-AND	36

LIST OF LISTINGS

Chapter 1

Introduction

Tradizionali approcci di ingegneria del software mostrano limiti crescenti nel rispondere alle esigenze dinamiche del business moderno, e numerosi studi empirici suggeriscono che l'agilità rappresenta una delle soluzioni più promettenti per superarli [AWSR03b, DD08a]. I metodi Agile, tra cui *Scrum* ed *eXtreme Programming* (XP), si fondano sui dodici principi del Manifesto Agile [B⁺01] e sono ormai adottati da un numero sempre maggiore di aziende allo scopo di rendere lo sviluppo software più rapido, adattivo e focalizzato sul valore. In particolare, il metodo Scrum è stato ampiamente discusso in letteratura [Sch95], che ne descrive le idee fondamentali e il ciclo di vita operativo.

Un elemento comune ai metodi Agile è la progettazione e implementazione incrementale, in cui il software è suddiviso in funzionalità specifiche (*user stories*) e, ad ogni iterazione (*sprint*), il team è chiamato a fornire il set di user stories che massimizza il valore per l'utente, soddisfacendo al tempo stesso vincoli di durata, dipendenza e relazioni tra le storie. Secondo il principio della *team awareness*, la pianificazione dello sprint si basa sulla condivisione e la mediazione delle stime fornite dai membri del team riguardo complessità delle storie, utilità, rischio di mancata consegna e dipendenze. Avanzare storie ad alto valore può anticipare risultati significativi per l'utente, rafforzando la consapevolezza del team; analogamente, anticipare storie critiche consente di limitare rischi di impatto tardivo, bilanciando però una maggiore probabilità di ritardi nelle fasi iniziali. Il contributo di [Coh04a] offre indicazioni pratiche per valutare complessità e valore delle

user stories.

Una pianificazione di sprint efficace è unanimemente riconosciuta come fattore chiave per il successo di un progetto agile [Coh05]. L'efficacia del piano dipende sia dalla precisione delle stime (aspetto legato all'esperienza del team), sia dalla capacità di considerare correttamente variabili e vincoli progettuali: quest'ultimo obiettivo si traduce in un problema di ottimizzazione la cui complessità cresce con la dimensione del progetto, e il cui mancato raggiungimento può generare inefficienze, extracosti e ritardi.

Questa tesi prende come riferimento il lavoro [BGRT14], su cui vengono sviluppate e approfondite nuove tecniche di ottimizzazione e strumenti matematici per il supporto alla sprint planning in ambiente Agile.

Chapter 2

Descrizione del problema

Poiché la soddisfazione del cliente rappresenta uno dei principali indicatori di successo di un progetto, i metodi agili si propongono di rispondere in modo più efficace alle esigenze degli utenti, riducendo i tempi di consegna e aumentando la flessibilità del processo di sviluppo. Accelerare il time-to-market consente infatti di ridurre la pressione competitiva, mentre la flessibilità permette di adattarsi rapidamente sia ai progressi tecnologici sia ai cambiamenti nei requisiti degli utenti. Per conseguire tali obiettivi, l'approccio agile adotta una serie di pratiche complementari, tra cui lo sviluppo incrementale e iterativo, il coinvolgimento continuo degli utenti, la consapevolezza del team, e una documentazione leggera ma efficace. Dall'esperienza dei professionisti del settore sono nati diversi framework agili basati su questi principi. Tra questi, Scrum e gli approcci ibridi, che combinano pratiche di Scrum ed eXtreme Programming (XP), risultano i più diffusi nelle aziende: Scrum si focalizza sugli aspetti organizzativi e di gestione del processo, mentre XP introduce pratiche tecniche specifiche, come la programmazione in coppia, per migliorare la qualità e l'efficienza dello sviluppo del codice.

Il ciclo di vita di un progetto Scrum è suddiviso in varie fasi. Durante la definizione delle user story, il team di sviluppo e gli utenti collaborano per delineare la struttura generale del sistema e individuare un insieme di funzionalità. Una user story rappresenta una funzionalità di piccole dimensioni ma di elevato valore per l'utente [Coh04b], descritta in modo semplice e sintetico. Essa costituisce una specifica leggera, che può essere ulteriormente approfondita attraverso un

dialogo continuo con l'utente, ma deve al contempo contenere informazioni sufficienti a permettere al team di stimarne la complessità di sviluppo e pianificarne l'implementazione in modo efficace.

Durante la fase di stima delle user story, a ciascuna storia vengono attribuiti due principali parametri: l'utilità e la complessità. Le valutazioni possono essere effettuate sulla base di esperienze pregresse, pareri di esperti o tramite tecniche basate sul consenso come il planning poker. L'utilità rappresenta il valore per l'azienda percepito dall'utente che ha definito la user story. In alcuni casi è sufficiente stabilire un ordine di priorità tra le storie, mentre in altri l'utilità viene quantificata mediante un punteggio numerico. La complessità di sviluppo, invece, viene espressa in story point, un'unità adimensionale che viene preferita rispetto a misure di tempo o risorse per ridurre la soggettività e favorire la comparabilità tra le stime. L'assegnazione degli story point è effettuata dai membri del team in base alla loro esperienza, alla conoscenza del dominio e alle caratteristiche specifiche del progetto. Sono utilizzabili diverse tecniche di dimensionamento [Coh05] tra cui l'utilizzo di scale numeriche, come la sequenza di Fibonacci, intervalli di valori o l'adozione di sistemi che si basano su categorie qualitative, come l'utilizzo delle taglie delle magliette.

Durante la fase di prioritizzazione delle user story, il team assegna a ciascuna storia un livello di priorità basato principalmente sulla sua utilità e sul rischio associato, oltre a identificare le eventuali dipendenze tra le diverse storie. Vi sono due categorie di fattori di rischio, vi sono le storie critiche: si tratta di user story che esercitano un forte impatto sulle altre e rappresentano componenti fondamentali del progetto. Una scelta errata nell'assegnazione o nella stima di queste storie può compromettere il successo complessivo dello sprint e dei cicli successivi, creando effetti a cascata su altre dipendenze. E vi sono le storie incerte, caratterizzate da un elevato grado di difficoltà nella stima del tempo e dello sforzo richiesto. Questa incertezza nasce da potenziali imprevisti, come cambiamenti nei requisiti espressi dall'utente, problemi tecnici inaspettati durante l'implementazione, o complessità che emergeranno solo durante lo sviluppo. Entrambe le categorie di rischio vengono generalmente rappresentate tramite valori numerici. Le dipendenze, invece,

esprimono vincoli di sequenza nello sviluppo, indicando che una user story può essere avviata solo dopo il completamento di una o più storie precedenti. Sebbene i metodi agili tendano a limitare le dipendenze per mantenere elevata la flessibilità del progetto, alcune di esse risultano inevitabili e devono essere rispettate per garantire la coerenza del processo di sviluppo.

L'elenco delle user story, una volta definite le priorità, costituisce il product backlog, che viene poi suddiviso in sprint durante la fase di pianificazione dello sprint. Gli sprint devono avere una durata breve e costante, solitamente compresa tra una e le quattro settimane, in modo da assicurare un feedback rapido e continuo da parte degli utenti. L'assegnazione delle user story agli sprint si basa su tre elementi fondamentali: la velocità di sviluppo del team, la complessità delle singole story e le possibili correlazioni o affinità tra le story. La velocità di sviluppo rappresenta il numero stimato di story point, che rappresenta l'unità minima per stimare lo sforzo necessario a completare una user story o un'attività, che il team è in grado di completare giornalmente. Questo valore viene utilizzato per calcolare la capacità dello sprint, ovvero il numero massimo di story point che il team può realisticamente erogare durante l'intera durata dello sprint. Infine, l'affinità tra story si riferisce alle correlazioni logiche o funzionali tra user story che, se incluse nello stesso sprint, generano un valore maggiore per l'utente finale. Quando story correlate vengono realizzate insieme, gli utenti percepiscono più chiaramente il valore complessivo della funzionalità erogata e possono sperimentare un'esperienza più coerente e completa. Considerare l'affinità nella pianificazione permette quindi di massimizzare l'utilità percepita e di ottimizzare l'impatto di ogni rilascio incrementale.

Durante la fase di sviluppo dello sprint gli sviluppatori eseguono un'analisi dettagliata delle user story selezionate e producono lo sprint backlog, ovvero l'elenco operativo delle attività da completare. Successivamente, ogni membro del team si occupa di un sottoinsieme di story, seguendo l'intero ciclo di progettazione, implementazione e test. Al termine dello sprint si svolge la sprint review, durante la quale gli utenti e gli stakeholder esaminano le story completate per verificare che le funzionalità implementate corrispondano effettivamente ai requisiti richiesti.

Le story che superano la revisione vengono considerate completate e consegnate agli utenti come incremento del prodotto, mentre quelle non approvate vengono reinserite nel product backlog. In questa fase conclusiva, il team conduce anche una retrospettiva, analizzando i problemi riscontrati durante lo sprint e le soluzioni adottate, al fine di identificare opportunità di miglioramento per le iterazioni successive. Questo può includere, ad esempio, l'aggiornamento delle stime sulla base dell'esperienza maturata. Se dal feedback degli utenti emergono nuove user story o modifiche significative a quelle esistenti, oppure se le stime vengono riviste in modo sostanziale, viene eseguita una nuova fase di prioritizzazione del backlog prima dell'avvio dello sprint successivo, garantendo così che la pianificazione rimanga sempre allineata alle reali esigenze del progetto.

Il problema della pianificazione dello sprint può essere descritto come dato un insieme di user story e un insieme di sprint, allocare ogni story a uno sprint in modo da soddisfare tutti i vincoli relativi alla capacità dello sprint e alle dipendenze tra le storie e raggiungere anche come obiettivi la soddisfazione del cliente, che può essere raggiunta fornendo prima le user story con maggiore utilità per aumentare la consapevolezza e la fiducia dell'utente e includendo storie affini nello stesso sprint per aumentarne il valore per gli utenti, la gestione del rischio ottenibile promuovendo le user story critiche consentendo di identificare tempestivamente eventuali problemi o effetti collaterali che potrebbero propagarsi ad altre funzionalità, evitando impatti negativi tardivi e distribuendo le user story incerte in diversi sprint e posticipandole riducendo il rischio che più incertezze si concentrino nello stesso sprint, minimizzando la probabilità di ritardi nella consegna e garantendo una maggiore stabilità e prevedibilità del ciclo di sviluppo.

Il problema della pianificazione dello sprint può essere formulato come un Generalized Assignment Problem (GAP) con vincoli laterali. Questo tipo di problema consiste nell'assegnare un insieme di compiti a un insieme di risorse con l'obiettivo di ottimizzare una funzione (nel nostro caso, massimizzare l'utilità totale), rispettando i limiti di capacità di ciascuna risorsa e soddisfacendo vincoli aggiuntivi che impongono condizioni specifiche sulle assegnazioni.

Nel contesto della pianificazione agile, gli sprint rappresentano le risorse (o

zaini), mentre le user story costituiscono i compiti (o elementi) da assegnare. Ogni user story è caratterizzata da due attributi fondamentali: gli story point, che misurano il peso dell'elemento in termini di sforzo e complessità richiesti, e l'utilità, che ne rappresenta il valore per il progetto e per l'utente finale. La capacità dello sprint corrisponde al limite di capacità della risorsa ed è determinata dal numero massimo di story point che il team può completare, calcolato in base alla durata prevista dello sprint e alla velocità di sviluppo del team stesso. Questa capacità rappresenta quindi il vincolo principale che regola quante e quali user story possono essere assegnate a ciascun sprint.

La funzione obiettivo da massimizzare è l'utilità cumulativa dell'intero progetto. L'utilità di ogni singola user story viene incrementata se alcune storie affini sono incluse nello stesso sprint, poiché questo aumenta il valore percepito dall'utente, e/o se la storia è classificata come critica, riconoscendone l'importanza strategica per il successo del progetto.

Per quanto riguarda la gestione del rischio, nella formulazione del vincolo di capacità, gli story point delle user story vengono aumentati in proporzione alla loro incertezza. Questo meccanismo penalizza l'inclusione di due o più storie incerte nello stesso sprint (obiettivo 2-ii), favorendo una loro distribuzione equilibrata su sprint diversi e riducendo così il rischio di ritardi nella consegna.

È importante sottolineare che il Problema di Assegnazione Generalizzato appartiene alla classe dei problemi NP-hard, il che significa che trovare una soluzione ottimale richiede tempi computazionali che crescono esponenzialmente con le dimensioni del problema, rendendo necessario l'impiego di algoritmi euristici o metaeuristici per progetti di scala reale.

Chapter 3

stato dell'arte

I principi Agile hanno registrato una diffusione crescente nell'ultimo decennio, stimolando lo sviluppo di numerosi approcci metodologici proposti tanto dai professionisti quanto dalla comunità di ricerca [AWSR03a]. Una revisione sistematica presentata in letteratura [DD08b] confronta diversi metodi Agile analizzandone sia le caratteristiche organizzative che quelle tecniche, evidenziando in particolare la crescente adozione delle pratiche Scrum e XP nei contesti industriali. L'approccio Scrum è approfondito ulteriormente in [Sch97], dove vengono descritti i fondamenti metodologici e il ciclo di vita caratteristico, contribuendo così alla comprensione dei meccanismi e delle peculiarità che contraddistinguono questo framework.

Per quanto concerne la stima del valore del software, la letteratura propone molteplici approcci senza tuttavia convergere verso un modello condiviso. Nel contesto più ampio dell'ingegneria del software basata sul valore, in [RFB09] viene introdotta una matrice di decomposizione del valore che integra tre aspetti del software (tecnologia, progettazione e artefatto) con tre componenti del valore (valore intrinseco, esternalità e valore dell'opzione), fornendo in ciascuna cella domande specifiche che guidano gli analisti nell'interpretazione delle diverse combinazioni. Alternativamente, in [PP04] viene proposto un modello per esprimere il valore del software in termini monetari, sfruttando la relazione tra fattori tecnologici e di mercato. Una sintesi più completa è fornita da [KGW13], che distinguono quattro prospettive di valore: finanziaria, cliente, processo aziendale interno e innovazione e apprendimento. Gli autori sviluppano una Software Value Map (SVM) in cui

ciascuna prospettiva è scomposta nelle sue componenti e sottocomponenti principali, offrendo agli analisti uno strumento per costruire una comprensione condivisa del valore. Sulla base della SVM, propongono inoltre un modello pratico articolato in tre fasi per ottenere la stima finale dell'utilità: selezione dello scenario caratterizzante il progetto specifico, identificazione del pattern di componenti di valore che meglio si adatta allo scenario selezionato, e definizione di una valutazione quantitativa del pattern attraverso la stima delle componenti individuate.

Nell'ampio contesto della pianificazione dei progetti, la maggior parte degli sforzi di ricerca degli ultimi anni si è concentrata sullo sviluppo di procedure esatte o euristiche per la generazione di una pianificazione di base praticabile in ambiente deterministico, con numerosi modelli e algoritmi proposti in letteratura come documentato nella panoramica di [KP01]. La complessità aumenta significativamente quando un team deve pianificare simultaneamente più progetti, situazione frequente nella pratica che richiede approcci specifici come evidenziato in [PSW94]. In questo scenario, [DB98] distingue due livelli di pianificazione: il primo livello, denominato pianificazione approssimativa della capacità, affronta il problema di pianificazione a medio termine, mentre il secondo livello, chiamato pianificazione di progetto con vincoli di risorse, si occupa della pianificazione operativa a breve termine. Questo approccio a due livelli, esplorato anche in [GS05], utilizza una scomposizione top-down delle attività in grandi pacchetti di lavoro per ridurre la complessità della pianificazione a medio termine.

Secondo le classificazioni dei problemi di pianificazione di progetto proposte in [HDDR99] e [BDM⁺99], il problema affrontato può essere classificato come vincolato dalle risorse, con risorse rinnovabili (quali la manodopera) disponibili periodo per periodo. Analogamente al modello PERT/CPM di base, vengono considerate precedenze di tipo finish-to-start con sfasamento temporale nullo e non è consentita alcuna prelazione sulle attività. La funzione obiettivo adottata differisce tuttavia dall'approccio tradizionale: anziché minimizzare la durata complessiva del progetto, si persegue una misura di completamento incrementale che mira a massimizzare il valore aziendale percepito dagli utenti, riflettendo una prospettiva orientata al valore piuttosto che alla sola efficienza temporale.

Il problema della pianificazione ha acquisito crescente interesse nell'ambito delle metodologie iterative e incrementali. In [DCH03] viene proposta una strategia di sviluppo software basata su fattori finanziari, applicabile in contesti iterativi, che genera una sequenza ottimale di requisiti massimizzando nel tempo il valore attuale netto, ovvero una combinazione di ricavi, costi e rischi di ciascun requisito. Gli autori presentano due strategie risolutive: un algoritmo greedy che seleziona il successivo requisito da soddisfare considerando quelli senza precursori non soddisfatti e con il massimo valore attuale netto, e un approccio look-ahead che estende la strategia greedy analizzando sottoinsiemi di sequenze di precedenza redditizie.

Un modello specificamente orientato ai metodi agili è descritto in [kS11], dove viene presentato un modello concettuale per la pianificazione dei rilasci e sviluppato un modello di ottimizzazione finalizzato all'assegnazione dei requisiti alle diverse iterazioni di un rilascio, con l'obiettivo di massimizzare l'utilità complessiva fornita pur tenendo conto delle precedenze e delle condizioni di accoppiamento. In questo contesto, gli autori illustrano un algoritmo branch-and-bound per la risoluzione del modello, che incorpora esplicitamente la gestione del rischio. Un ulteriore contributo rilevante nel contesto agile è quello di [vTdP11], focalizzato primariamente sulla gestione del rischio e dell'incertezza nei progetti XP. L'approccio proposto stima la velocità di sviluppo del team e considera più set di user story con rilevanza decrescente secondo la classificazione MoSCoW:

- *set must have* (requisiti indispensabili)
- *set should have* (requisiti importanti)
- *set could have* (requisiti desiderabili)

L'obiettivo è assegnare ciascuna user story al set più appropriato, massimizzando l'utilità complessiva e rispettando le precedenze e gli accoppiamenti tra le user story mediante un algoritmo branch-and-bound per identificare la soluzione ottimale. Tuttavia, il numero limitato di set considerati determina un piano a grana grossa che necessita di successivo affinamento per ottenere una pianificazione operativa, ad esempio suddividendo i set secondo limiti di budget o frammentando le user story in attività più granulari.

In un precedente lavoro è stato inoltre proposto un approccio per la pianificazione dello sprint di base nei progetti di data warehouse agile [GRT12]. Il modello presentato in quel contributo si distingue per una struttura più semplice rispetto a quella qui descritta, non includendo storie forzate e adottando una modellazione dell'accoppiamento meno espressiva; inoltre, il problema della ripianificazione non viene affrontato in quella formulazione.

Nei contesti reali, l'ambiente di progetto difficilmente può essere considerato deterministico a causa della varietà di eventi imprevisti che possono verificarsi e dell'imprecisione intrinseca delle stime, rendendo necessarie politiche di gestione del cambiamento. Sebbene nessuno dei lavori precedentemente menzionati si occupi specificamente di gestione del cambiamento, un approccio rilevante in questa direzione è rappresentato in [GR04], orientato a contesti iterativi e incrementali. In questo framework, un piano di rilascio include diversi incrementi e in ogni fase un insieme di requisiti viene assegnato agli incrementi attuali e futuri in modo da restituire il miglior compromesso tra le priorità degli stakeholder e i vincoli di sviluppo, quali capacità di incremento, precedenze e condizioni di accoppiamento. Il modello è formalizzato come un problema di zaino multiplo e risolto mediante un algoritmo genetico. Per gestire il cambiamento, Evolve implementa una strategia parziale di ripianificazione: a ogni incremento sono consentiti nuovi requisiti e modifiche alle priorità o ai vincoli, e una nuova soluzione viene generata ex novo.

Nel contesto specifico della pianificazione Scrum, in [LvdABD10] viene fornita una formulazione a zaino di un modello di ottimizzazione per la pianificazione a singola iterazione che seleziona i requisiti massimizzando il profitto dell'iterazione successiva e rispettando i vincoli di sviluppo. L'evoluzione viene gestita consentendo modifiche ai parametri dopo ogni iterazione e valutandone l'impatto sul modello, con una nuova soluzione per l'iterazione successiva generata da zero incorporando modifiche e storie aggiuntive. Un approccio più sofisticato è rappresentato dalla pianificazione bi-obiettivo proposta in [SR07], in cui l'iterazione successiva viene pianificata considerando l'impatto di nuovi requisiti o modifiche sul sistema esistente dal punto di vista aziendale e di sviluppo. Viene generato un insieme di piani alternativi, ciascuno dei quali riflette una diversa importanza

relativa degli aspetti aziendali e di implementazione; il piano ottimale viene quindi selezionato come quello che meglio soddisfa un gruppo di interdipendenze, denominate SD-coupling, tra i requisiti identificati attraverso l'analisi di impatto. Questo approccio permette di bilanciare in modo esplicito le esigenze di business con le complessità tecniche derivanti dall'evoluzione del sistema.

Sebbene questi approcci affrontino specificamente le problematiche relative all'incertezza e alla ripianificazione, nessuno di essi si occupa di evitare che il nuovo piano interferisca con quello precedente, aspetto che costituisce uno dei contributi distintivi di questo lavoro. Per identificare contributi in questa direzione, è necessario esaminare l'area di ricerca sulla pianificazione in condizioni di incertezza, i cui sforzi si sono concentrati su due fronti principali: la pianificazione proattiva e la pianificazione reattiva [DH02].

La pianificazione proattiva, o robusta, si concentra sullo sviluppo di una pianificazione di base che incorpora un certo grado di anticipazione della variabilità durante l'esecuzione del progetto al fine di proteggere la pianificazione stessa da perturbazioni. Ad esempio, in [HLD02] viene definita una pianificazione iniziale aggressiva a cui vengono successivamente aggiunti buffer di risorse e tempo che proteggono i percorsi critici. Oltre alle regole empiriche, come la regola del dimensionamento del buffer al 50%, metodi più sofisticati per il dimensionamento dei buffer sono discussi in [New98].

La schedulazione reattiva si riferisce invece alle modifiche che potrebbero dover essere apportate alla schedulazione durante l'esecuzione del progetto e può basarsi su diverse strategie. Da un lato, l'approccio reattivo può utilizzare tecniche semplici volte a ripristinare rapidamente la coerenza della schedulazione: ad esempio, la regola dello spostamento a destra proposta in [SOS93] posticipa tutte le attività interessate dall'interruzione della schedulazione. D'altro canto, un approccio di schedulazione reattiva può comportare una rischedulazione completa delle attività rimanenti. In caso di rischedulazione, la nuova schedulazione può differire notevolmente da quella di base, circostanza non auspicabile poiché annullerebbe gli impegni precedentemente stabiliti generando costi aggiuntivi, tensioni e insoddisfazione sia da parte dei clienti che dei membri del team. Per questo motivo, gli approcci di rischedulazione naïve si basano spesso su euristiche che effettuano

riorganizzazioni locali dei piani. In alternativa, la rischedulazione può adottare una strategia di minima perturbazione che mira alla stabilità ex post, basandosi su algoritmi esatti o subottimali il cui obiettivo è la minimizzazione di una funzione delle differenze tra gli istanti di inizio di ciascuna attività nella schedulazione nuova e in quella originale [SW00], oppure la minimizzazione del numero di attività da svolgere in sprint diversi [AA03].

Chapter 4

Formulazione matematica

In questa sezione definiamo matematicamente il problema di pianificazione agile. Consideriamo l'insieme $U = \{1, \dots, n\}$ degli indici delle n user story da realizzare durante il progetto. Ogni user story $j \in U$ è caratterizzata dai seguenti attributi principali:

- u_j : l'utilità, che rappresenta il valore apportato dalla user story;
- r_j^{cr} : il rischio critico, che misura l'impatto potenziale della story sulle altre;
- r_j^{unc} : l'incertezza, che quantifica la difficoltà di stima dovuta a possibili imprevisti;
- p_j : la complessità, espressa in story point, che indica lo sforzo necessario per completarla.

Definiamo inoltre $Y_j \subseteq U$ come l'insieme delle user story affini alla story j , ovvero quelle che, se assegnate allo stesso sprint, generano un valore aggiunto. Per ognuna di esse, a_j rappresenta l'incremento di utilità ottenuto per ogni user story affine assegnata allo stesso sprint. Nel caso in cui $Y_j = \emptyset$, poniamo $a_j = 0$.

Per gestire le dipendenze tra user story, distinguiamo due sottoinsiemi U^{OR} e U^{AND} , rispettivamente contenenti le user story con dipendenze di tipo OR e AND.

- Per ogni $j \in U^{OR}$, indichiamo con $D_j^{OR} \subseteq U$ l'insieme delle user story dalle quali j dipende secondo una logica OR: almeno una delle storie in D_j^{OR} deve

essere assegnata allo stesso sprint di j o a uno sprint precedente affinché j possa essere eseguita.

- Per ogni $j \in U^{AND}$, definiamo $D_j^{AND} \subseteq U$ come l'insieme delle user story da cui j dipende in modalità AND: tutte le storie in D_j^{AND} devono essere assegnate allo stesso sprint di j o a sprint precedenti.

Consideriamo infine l'insieme $S = \{1, \dots, m\}$ degli m sprint pianificati per il progetto, dove ogni sprint $i \in S$ ha una capacità massima p_i^{max} , espressa in story point, che rappresenta il limite di lavoro che il team può completare durante quello sprint.

Le variabili decisionali che modellano il problema sono:

- $x_{ij} \in \{0, 1\}$, variabile binaria che vale 1 se la user story j viene assegnata allo sprint i , 0 altrimenti;
- $y_{ij} \in \mathbb{Z}_{\geq 0}$, variabile intera non negativa che indica il numero di user story affini a j (cioè appartenenti a Y_j) assegnate allo stesso sprint i .

Il modello di programmazione lineare intera mista è:

$$z_P = \max \sum_{k=1}^m \sum_{i=1}^k \sum_{j=1}^n u_j (r_j^{cr} x_{ij} + a_j y_{ij}) \quad (4.1)$$

$$s.t. \sum_{j=1}^n p_j r_j^{un} x_{ij} \leq p_i^{max}, \quad i \in S \quad (4.2)$$

$$\sum_{i=1}^m x_{ij} = 1, \quad j \in U \quad (4.3)$$

$$\sum_{k=1}^i \sum_{z \in D_j^{OR}} x_{kz} \geq x_{ij}, \quad i \in S, j \in U^{OR} \quad (4.4)$$

$$\sum_{k=1}^i \sum_{z \in D_j^{AND}} x_{kz} \geq x_{ij} |D_j^{AND}|, \quad i \in S, j \in U^{AND} \quad (4.5)$$

$$y_{ij} \leq \sum_{k \in Y_j} x_{ik}, \quad i \in S, j \in U \quad (4.6)$$

$$y_{ij} \leq |Y_j| x_{ij}, \quad i \in S, j \in U \quad (4.7)$$

$$x_{ij} \in \{0, 1\}, \quad i \in S, j \in U \quad (4.8)$$

$$y_{ij} \geq 0, \quad i \in S, j \in U \quad (4.9)$$

Ecco il testo rielaborato in formato LaTeX:

La funzione obiettivo (4.1) ha lo scopo di massimizzare l'utilità complessiva del progetto agile. L'utilità base u_j di ciascuna user story j viene incrementata considerando due fattori principali:

1. il **rischio di criticità** r_j^{cr} , che assegna un peso maggiore alle storie critiche e ne incentiva l'esecuzione anticipata negli sprint iniziali, riducendo il rischio di impatti tardivi sul progetto;
2. l'**incremento di affinità** a_j , che aumenta il valore percepito dall'utente quando storie correlate vengono sviluppate congiuntamente nello stesso sprint, migliorando la coerenza funzionale del rilascio.

Per ogni user story j , il numero di storie affini che vengono incluse nello sprint

i è quantificato dalla variabile y_{ij} , calcolata come:

$$y_{ij} = \sum_{k \in Y_j} x_{ik},$$

dove $y_{ij} = 0$ quando $x_{ij} = 0$, ovvero quando la story j non è assegnata allo sprint i . Trattandosi di un problema di massimizzazione, i vincoli (4.6) e (4.7) garantiscono il corretto calcolo delle variabili y_{ij} senza la necessità di imporre vincoli espliciti di integralità su di esse.

I **vincoli di capacità** (4.2) assicurano che la complessità complessiva delle user story assegnate a ciascuno sprint, misurata in story point, non ecceda la capacità massima disponibile $p_i^{\max}$ per quello sprint, rispettando così i limiti operativi del team.

I **vincoli di assegnazione** (4.3) impongono che ogni user story sia assegnata esattamente a uno e un solo sprint, garantendo che nessuna storia venga trascurata o duplicata nella pianificazione.

Le **relazioni di dipendenza** tra user story sono modellate attraverso i vincoli (4.4) e (4.5), che regolano l'ordine di esecuzione delle storie in base ai loro prerequisiti:

- per le dipendenze di tipo **OR**, il vincolo (4.4) permette l'assegnazione della story j allo sprint i solamente se *almeno una* delle user story appartenenti all'insieme D_j^{OR} è stata completata in uno sprint precedente o nello stesso sprint ($i' \leq i$);
- per le dipendenze di tipo **AND**, il vincolo (4.5) richiede invece che *tutte* le user story contenute nell'insieme D_j^{AND} siano state completate entro lo sprint i , assicurando che tutti i prerequisiti necessari siano soddisfatti prima di procedere con l'esecuzione di j .

4.1 Procedure di riduzione

Le procedure di riduzione mirano a rafforzare i vincoli di capacità (4.2) modificando le capacità degli sprint o i pesi $pr_j = p_j r_j^{un}$ delle user story. Approcci simili sono utilizzati per problemi di packing in [BHM02, BM03, BM10].

4.1.1 Modifica delle capacità degli sprint

Se non esiste alcuna combinazione di user story che saturi esattamente la capacità p_i^{max} dello sprint $i \in S$, è possibile ridurre tale capacità rimuovendo i "punti storia inutilizzabili" senza alterare il valore ottimale del problema. La nuova capacità può essere trovata risolvendo il problema di Subset Sum:

$$p_i^{max} = \max \left\{ \sum_{k \in U} pr_k \xi_k : \sum_{k \in U} pr_k \xi_k \leq p_i^{max}, \quad \xi_j \in \{0, 1\}, j \in U \right\}$$

dove $pr_j = p_j r_j^{un}$ è il peso effettivo associato alla user story j . Il problema di Subset Sum può essere efficacemente risolto utilizzando una procedura di programmazione dinamica.

4.1.2 Modifica dei pesi delle user story

Un ulteriore miglioramento consiste nell'aumentare il peso pr_j di una user story $j \in U$, mantenendo la fattibilità dello sprint che la contiene. Per ogni sprint $i \in S$, definiamo il valore p_{ij}'' :

$$p_{ij}'' = \max \left\{ \sum_{h \in U} pr_h \xi_h : \sum_{h \in U} pr_h \xi_h \leq p_i^{max}, \quad \xi_j = 1, \quad \xi_k \in \{0, 1\}, k \in U \setminus \{j\} \right\}$$

ossia la massima somma di pesi che include necessariamente la storia j senza superare la capacità dello sprint. In tal modo, è possibile aggiornare il peso di j come:

$$pr_j := pr_j + (p_i^{max} - p_{ij}'').$$

Per massimizzare il numero di storie a cui aggiornare il peso, si propone di ordinare le user story in ordine decrescente di peso, ovvero:

$$pr_1 \geq pr_2 \geq \dots \geq pr_n.$$

Queste tecniche di riduzione sfruttano problemi classici come il Subset Sum, risolvibili con algoritmi di programmazione dinamica, contribuendo a semplificare il modello e a ridurre i tempi di calcolo necessari per la soluzione ottimale.

Chapter 5

Euristiche per la pianificazione degli sprint

5.1 Euristiche greedy e di scambio del lavoro precedente

In [BGRT14] sono state proposte due euristiche greedy per costruire un piano di sprint e una procedura di post ottimizzazione basata su scambi locali. In questa sezione ne riprendiamo brevemente il funzionamento, poiché tali euristiche costituiscono la base anche per il lavoro sviluppato in questa tesi.

5.1.1 GreedyHeuristic: costruzione sprint-per-sprint

L'idea di base della *GreedyHeuristic* è la seguente: il piano viene costruito iterativamente, ottimizzando uno sprint per volta in ordine cronologico (prima lo sprint 1, poi lo sprint 2, e così via). A ogni passo, fissato uno sprint $i \in S$ e l'insieme F_i delle user story già assegnate agli sprint precedenti, si risolve il sottoproblema

(SP_i) :

$$(SP_i) \quad z_{SP_i} = \max \sum_{j=1}^n (m - i + 1) u_j (r_j^{cr} x_{ij} + a_j y_{ij}) \quad (5.1)$$

$$s.t. \quad \sum_{j=1}^n p_j r_j^{un} x_{ij} \leq p_i^{max} \quad (5.2)$$

$$\sum_{k=1}^i \sum_{z \in D_j^{OR}} x_{kz} \geq x_{ij} - |D_j \cap F_i|, \quad j \in U^{OR} \quad (5.3)$$

$$\sum_{k=1}^i \sum_{z \in D_j^{AND}} x_{kz} \geq x_{ij} |D_j| - |D_j \cap F_i|, \quad j \in U^{AND} \quad (5.4)$$

$$y_{ij} \leq \sum_{k \in Y_j} x_{ik}, \quad j \in U \quad (5.5)$$

$$y_{ij} \leq |Y_j| x_{ij}, \quad j \in U \quad (5.6)$$

$$x_{ij} \in \{0, 1\}, \quad j \in U \setminus F_i \quad (5.7)$$

$$x_{ij} = 0, \quad j \in F_i \quad (5.8)$$

$$y_{ij} \geq 0, \quad i \in S, j \in U \quad (5.9)$$

dove F_i è l'insieme delle storie già assegnate agli sprint $1, \dots, i-1$ (con $F_1 = \emptyset$).

Listing 5.1: GreedyHeuristic

```

1 Set $F_1 = \emptyset$, $i = 1$
2
3 WHILE ($F_i \neq U$ and $i \leq m$):
4     Compute the optimal solution $\mathbf{x}^*$ of
5     subproblem $SP_i$
6     Set $F_{i+1} = F_i \cup \{ j \in U : x^*_{ij} = 1 \}$
7     Set $i = i + 1$

```

Pseudocodice della GreedyHeuristic. Questa procedura è concettualmente greedy perché, a ogni passo, prende la “miglior scelta locale” per lo sprint corrente

(risolvendo SP_i) e non rivede le decisioni sugli sprint precedenti.

5.1.2 QuickGreedyHeuristic: versione veloce basata su knapsack

l'euristica *GreedyHeuristic*, riassunta in Figura 5.1, è estremamente rapida se eseguita una sola volta, ma il suo costo computazionale può diventare rilevante quando deve essere richiamata ripetutamente. In particolare come mostrato in [BGRT14], per alcune istanze di grandi dimensioni, una singola esecuzione di *GreedyHeuristic* richiede più di un secondo; di conseguenza, ripeterla per migliaia di iterazioni comporta un tempo totale dell'ordine di migliaia di secondi, imputabile unicamente alla componente greedy.

Per ridurre questo onere, proponiamo una variante accelerata dell'euristica greedy, denominata *QuickGreedyHeuristic*. L'idea di base consiste nel rilassare il sottoproblema SP_i rimuovendo i vincoli (5.3) (5.6). Il sottoproblema risultante, indicato con SP'_i , si riconduce a un problema di knapsack che può essere risolto in modo efficiente tramite programmazione dinamica. Tale rilassamento introduce tuttavia due criticità:

- in assenza dei vincoli di collegamento (5.5) (5.6), le variabili $\{y_{ij}\}$ diventano indipendenti dalle decisioni di assegnazione;
- l'eliminazione dei vincoli di dipendenza (5.3) (5.4) può determinare violazioni delle relazioni di precedenza tra le user story.

Per risolvere il primo problema, il knapsack SP'_i viene risolto ignorando le variabili $\{y_{ij}\}$, che sono poi valutate a posteriori a partire dalla soluzione $\{x_{ij}\}$ di SP'_i . In particolare, per ogni sprint i e user story j , poniamo

$$y_{ij} = \begin{cases} \sum_{k \in Y_j} x_{ik} & \text{se } x_{ij} = 1, \\ 0 & \text{altrimenti.} \end{cases}$$

In questo modo, le variabili y_{ij} riflettono il numero di user story “di supporto” selezionate nello sprint i a fronte della scelta di j .

Per quanto riguarda le dipendenze, la soluzione del knapsack rilassato SP'_i può violare i vincoli (5.3) (5.4). Fissato lo sprint corrente i e una user story j che viola almeno un vincolo di dipendenza, *QuickGreedyHeuristic* adotta una tra quattro strategie di recupero della fattibilità:

1. **Esclusione di j .** Si proibisce l'uso di j nello sprint i fissando $x_{ij} = 0$ e si riottimizza il knapsack SP'_i .
2. **Inserimento mirato di una dipendenza.** Si seleziona una user story $j' \in D_j^{OR}$ (oppure $j' \in D_j^{AND}$, a seconda del vincolo violato), non presente nella soluzione corrente di SP'_i , che massimizza il rapporto

$$\frac{u_{j'} r_{j'}^{cr}}{p_{j'} r_{j'}^{un}},$$

si fissa $x_{ij'} = 1$ e si riottimizza SP'_i .

3. **Inserimento completo per vincoli AND.** Solo nel caso di dipendenze di tipo AND, si fissano in soluzione tutte le user story $j' \in D_j^{AND}$ non selezionate (ponendo $x_{ij'} = 1$) e si riottimizza SP'_i ; per le dipendenze di tipo OR si applica invece la strategia (ii).
4. **Rinforzo progressivo dei profitti.** Per ogni user story j si introduce un coefficiente κ_j che moltiplica il profitto in ogni knapsack SP'_i . Per ogni $j' \in D_j^{OR} \setminus F_i$ (oppure $j' \in D_j^{AND} \setminus F_i$, a seconda del vincolo violato) con $x_{ij'} = 0$ nella soluzione corrente, si aumenta $\kappa_{j'}$ secondo una delle due regole:

$$(a) \quad \kappa_{j'} = \rho \kappa_{j'},$$

$$(b) \quad \kappa_{j'} = \kappa_{j'}^2.$$

Successivamente, si riavvia l'euristica greedy dal primo sprint, ponendo $i = 1$ e $F_1 = \emptyset$.

Listing 5.2: QuickGreedyHeuristic

```

1 Set z_star = -infinity
2
3 FOR each Strategy s = 1, 2, 3, 4:
4     Set z_temp = 0
5     Set F_1 = empty
6     Set i = 1
7     Set Iter = 0
8     WHILE (F_i != U AND i <= m):
9         REPEAT:
10             Compute optimal solution x_temp of knapsack
                SP_i_prime
11
12             IF (x_temp violates some dependency):
13                 Apply strategy s
14                 IF (s == 4):
15                     Set z_temp = 0
16                     Set F_1 = empty
17                     Set i = 1
18                     Set Iter = Iter + 1
19             UNTIL (x_temp feasible OR SP_i_prime infeasible OR
                Iter > MaxIter)
20             IF (SP_i_prime infeasible OR Iter > MaxIter):
21                 Set z_temp = -infinity
22                 Set i = m + 1
23             ELSE:
24                 z_temp = z_temp + value(SP_i)
25                 F_{i+1} = F_i union { j in U with x_temp[i,j] =
                    1 }
26                 i = i + 1
27             IF (z_star < z_temp):
28                 z_star = z_temp
29                 x_star = x_temp
30 END FOR

```

La Figura 5.2 riassume il funzionamento dell'algoritmo *QuickGreedyHeuristic*. Per ciascuna strategia $s = 1, 2, 3, 4$, l'algoritmo costruisce iterativamente una soluzione, aggiornando lo sprint corrente i , l'insieme delle user story già assegnate agli sprint precedenti (F_i) e il valore complessivo z' della soluzione ottenuta. Alla fine, viene mantenuta la migliore soluzione trovata $(\mathbf{x}^*, z^*)$.

In termini di proprietà, la strategia (i) garantisce sempre la convergenza a una soluzione fattibile, poiché ogni violazione viene gestita tramite esclusione della user story responsabile. Al contrario, le strategie (ii) e (iii), in particolare la terza, possono generare istanze di knapsack infeasibili (ad esempio quando la capacità di sprint è insufficiente per ospitare tutte le dipendenze forzate). In tali casi, se SP'_i risulta infeasibile, l'algoritmo interrompe il trattamento dello sprint i e passa all'iterazione successiva della procedura greedy. La strategia (iv) può richiedere un numero elevato di iterazioni prima di raggiungere la fattibilità; per questo motivo, viene imposto un numero massimo di iterazioni $MaxIter$, oltre il quale il tentativo con la strategia (iv) viene interrotto.

Nella fase sperimentale, si è fissato $MaxIter = 1000$. La strategia (iv) è stata applicata una volta utilizzando la regola (b), inizializzando i coefficienti a $\kappa_j = 1.025$ per ogni $j \in U$, e due volte utilizzando la regola (a), con $\kappa_j = 1$ per ogni $j \in U$ e valori di ρ pari a 2 e 5. La scelta di κ_j e ρ riflette un compromesso tra tempo di raggiungimento della fattibilità e qualità della soluzione: se i profitti crescono troppo rapidamente, si ottiene spesso una soluzione fattibile in poche iterazioni, ma potenzialmente di bassa qualità, poiché le user story “rinforzate” vengono anticipate ai primi sprint senza considerare adeguatamente la loro utilità reale; al contrario, se i profitti aumentano troppo lentamente, il numero di iterazioni necessario per ottenere una soluzione fattibile può diventare eccessivo.

5.1.3 ExchangeHeuristic: miglioramento locale per scambi

Sia *GreedyHeuristic* sia *QuickGreedyHeuristic* possono terminare senza individuare una soluzione fattibile, poiché tutti gli sprint risultano esaminati (cioè $i > m$) ma non tutte le user story sono assegnate agli sprint disponibili (ovvero $F_i \neq U$). Inoltre, anche quando *QuickGreedyHeuristic* produce una soluzione fattibile, tale soluzione può in genere essere ulteriormente migliorata: le strategie di recupero

della fattibilità rispetto ai vincoli di dipendenza, infatti, possono condurre a configurazioni non localmente ottimali.

In [BGRT14] viene proposta *ExchangeHeuristic* descritta in Figura 5.3. Data una soluzione $\mathbf{x}'$ da migliorare, la procedura tenta sistematicamente di scambiare user story tra coppie di sprint, ripetendo il processo fino a quando avviene almeno uno scambio migliorativo. In particolare, *ExchangeHeuristic* considera tre tipi di mosse:

1-0: spostamento di una user story $j \in U$ eseguita nello sprint i' verso uno sprint i ;

1-1: scambio di una user story $j \in U$ eseguita nello sprint i con una user story $j' \in U$ eseguita nello sprint i' ;

2-1: scambio di due user story $j, j' \in U$ eseguite nello sprint i con una user story $j'' \in U$ eseguita nello sprint i' .

Uno scambio viene effettivamente applicato solo se è contemporaneamente *fattibile* e *profittevole*. La fattibilità richiede che, dopo lo scambio, i vincoli di capacità degli sprint coinvolti rimangano soddisfatti e che non si generino violazioni delle dipendenze tra user story; la profittabilità richiede invece che il valore complessivo della funzione obiettivo aumenti.

L'algoritmo *ExchangeHeuristic* può essere esteso includendo mosse di scambio più complesse (ad esempio scambi k - ℓ con $k, \ell > 2$). Tuttavia, l'incremento di complessità computazionale associato a tali estensioni tende a non essere compensato da miglioramenti significativi della qualità della soluzione, motivo per cui in questo lavoro ci si limita alle mosse 1-0, 1-1 e 2-1 descritte sopra.

Listing 5.3: ExchangeHeuristic

```

1 Repeat
2   // Apply 1-0 exchanges
3   For sprint i = 1..m-1:
4     For sprint i' = i+1..m:
5       For each story j with x'[i',j] = 1:
6         If moving j from i' to i is feasible and
           profitable:
7           x'[i',j] = 0
8           x'[i,j] = 1
9   // Apply 1-1 exchanges
10  For sprint i = 1..m-1:
11    For sprint i' = i+1..m:
12      For each pair j, j' with x'[i,j]=1 and x'[i',j
        ']=1:
13        If exchanging j and j' is feasible and
           profitable:
14          x'[i,j] = 0
15          x'[i',j'] = 0
16          x'[i,j'] = 1
17          x'[i',j] = 1
18  // Apply 2-1 exchanges
19  For sprint i = 1..m:
20    For sprint i' = 1..m:
21      For each triple j, j', j'' with x'[i,j]=x'[i,j
        ']=x'[i',j'']=1:
22        If exchanging j with {j',j''} is feasible
           and profitable:
23          x'[i,j] = 0
24          x'[i,j'] = 0
25          x'[i',j'']= 0
26          x'[i',j] = 1
27          x'[i',j'] = 1
28          x'[i,j''] = 1
29 Until no exchanges occur.

```


5.2 A Lagrangian Heuristic

La letteratura propone numerose euristiche basate su metodi di decomposizione e rilassamento lagrangiano; introduzioni esaustive a questi temi sono disponibili, tra gli altri, in [?, ?, ?]. In [BGRT14] si descrive una procedura euristica che sfrutta il rilassamento lagrangiano del modello, combinato con un algoritmo del sottogradient e con le euristiche greedy introdotte in precedenza.

Il rilassamento lagrangiano si ottiene dualizzando i vincoli (4.3)–(4.7) mediante i vettori di penalità $\{\lambda_j\}$, $\{\lambda_{ij}^{OR}\}$, $\{\lambda_{ij}^{AND}\}$, $\{\lambda_{ij}^{Y1}\}$ e $\{\lambda_{ij}^{Y2}\}$. Le penalità λ_j , $j \in U$, sono non vincolate, mentre le restanti sono non positive, così da interpretare le violazioni dei vincoli corrispondenti come costi aggiuntivi nel problema rilassato. Il problema lagrangiano risultante è:

$$(LR) \quad z_{LR}(\boldsymbol{\lambda}) = \max \sum_{k=1}^m \sum_{i=1}^k \sum_{j=1}^n (u'_{ij}(\boldsymbol{\lambda})x_{ij} + u''_{ij}(\boldsymbol{\lambda})y_{ij}) + \sum_{j=1}^n \lambda_j \quad (5.10)$$

$$\text{s.t.} \quad \sum_{j=1}^n p_j r_j^{un} x_{ij} \leq p_i^{\max}, \quad i \in S \quad (5.11)$$

$$x_{ij} \in \{0, 1\}, \quad i \in S, j \in U \quad (5.12)$$

$$0 \leq y_{ij} \leq |Y_j|, \quad i \in S, j \in U \quad (5.13)$$

dove le *utilità penalizzate* $u'_{ij}(\boldsymbol{\lambda})$ e $u''_{ij}(\boldsymbol{\lambda})$ sono definite da

$$\begin{aligned} u'_{ij}(\boldsymbol{\lambda}) &= u_j r_j^{cr} - \lambda_j + \left(\lambda_{ij}^{OR} - \sum_{k=i}^m \sum_{j' \in \bar{D}_j^{OR}} \lambda_{kj'}^{OR} \right) + \\ &\quad + \left(|D_j^{AND}| \lambda_{ij}^{AND} - \sum_{k=i}^m \sum_{j' \in \bar{D}_j^{AND}} \lambda_{kj'}^{AND} \right) - \lambda_{ij}^{Y1} - |Y_j| \lambda_{ij}^{Y2} \\ u''_{ij}(\boldsymbol{\lambda}) &= u_j a_j + \lambda_{ij}^{Y1} + \lambda_{ij}^{Y2}, \end{aligned} \quad (5.14)$$

con $\bar{D}_j^{OR} = \{j' \in U : j \in D_{j'}^{OR}\}$ e $\bar{D}_j^{AND} = \{j' \in U : j \in D_{j'}^{AND}\}$.

Grazie alla struttura dei vincoli di capacità, il problema LR si decompone in

$2m$ sottoproblemi indipendenti, due per ogni sprint $i \in S$:

$$(LR_i^1) \quad z_{LR_i^1}(\boldsymbol{\lambda}) = \max \sum_{j=1}^n u'_{ij}(\boldsymbol{\lambda}) x_{ij} \quad (5.15)$$

$$\text{s.t.} \quad \sum_{j=1}^n p_j r_j^{un} x_{ij} \leq p_i^{\max} \quad (5.16)$$

$$x_{ij} \in \{0, 1\}, \quad j \in U \quad (5.17)$$

e

$$(LR_i^2) \quad z_{LR_i^2}(\boldsymbol{\lambda}) = \max \sum_{j=1}^n u''_{ij}(\boldsymbol{\lambda}) y_{ij} \quad (5.18)$$

$$\text{s.t.} \quad 0 \leq y_{ij} \leq |Y_j|, \quad j \in U \quad (5.19)$$

Il sottoproblema LR_i^1 è un problema di knapsack, mentre LR_i^2 è risolvibile per ispezione, assegnando $y_{ij} = |Y_j|$ se $u''_{ij}(\boldsymbol{\lambda}) > 0$ e $y_{ij} = 0$ altrimenti. Il valore ottimo del rilassamento lagrangiano è quindi

$$z_{LR}(\boldsymbol{\lambda}) = \sum_{i=1}^m (m - i + 1) (z_{LR_i^1}(\boldsymbol{\lambda}) + z_{LR_i^2}(\boldsymbol{\lambda})) + \sum_{j=1}^n \lambda_j, \quad (5.20)$$

che fornisce un upper bound valido per il problema originale P [web:52][web:55]. La ricerca del vettore di penalità $\boldsymbol{\lambda}^*$ che minimizza $z_{LR}(\boldsymbol{\lambda})$ porta al *dual lagrangiano* $z_{LR}(\boldsymbol{\lambda}^*) = \min_{\boldsymbol{\lambda}} \{z_{LR}(\boldsymbol{\lambda})\}$, affrontato qui tramite un algoritmo del sottogradiente [web:55][web:59].

Sia $(\mathbf{x}, \mathbf{y})$ la soluzione, di valore $z_{LR}(\boldsymbol{\lambda})$, ottenuta risolvendo LR a una generica iterazione del sottogradiente. I moltiplicatori lagrangiani vengono aggiornati

secondo

$$\begin{aligned}
 \lambda_j &= \lambda_j + \alpha g_j, & j \in U \\
 \lambda_{ij}^{OR} &= \max\{0, \lambda_{ij}^{OR} + \alpha g_{ij}^{OR}\}, & i \in S, j \in U^{OR} \\
 \lambda_{ij}^{AND} &= \max\{0, \lambda_{ij}^{AND} + \alpha g_{ij}^{AND}\}, & i \in S, j \in U^{AND} \\
 \lambda_{ij}^{Y1} &= \max\{0, \lambda_{ij}^{Y1} + \alpha g_{ij}^{Y1}\}, & i \in S, j \in U \\
 \lambda_{ij}^{Y2} &= \max\{0, \lambda_{ij}^{Y2} + \alpha g_{ij}^{Y2}\}, & i \in S, j \in U
 \end{aligned} \tag{5.21}$$

dove α è la lunghezza del passo lungo la direzione di ricerca data dal sottogradiente $\mathbf{g}$, le cui componenti sono

$$\begin{aligned}
 g_j &= \sum_{i=1}^m x_{ij} - 1, & j \in U \\
 g_{ij}^{OR} &= \sum_{k=1}^i \sum_{z \in D_j^{OR}} x_{kz} - x_{ij}, & i \in S, j \in U^{OR} \\
 g_{ij}^{AND} &= \sum_{k=1}^i \sum_{z \in D_j^{AND}} x_{kz} - x_{ij} |D_j^{AND}|, & i \in S, j \in U^{AND} \\
 g_{ij}^{Y1} &= \sum_{k \in Y_j} x_{ik} - y_{ij}, & i \in S, j \in U \\
 g_{ij}^{Y2} &= |Y_j| x_{ij} - y_{ij}, & i \in S, j \in U
 \end{aligned} \tag{5.22}$$

La regola per il passo adottata negli esperimenti è

$$\alpha = \beta \frac{0.1 z_{LR}(\boldsymbol{\lambda})}{\|\mathbf{g}\|_2^2},$$

dove β è inizializzato a un valore dipendente dal problema (nel nostro caso, $\beta = 3$) e viene ridotto moltiplicandolo per 0.85 se, dopo un numero prefissato di iterazioni (10), il valore $z_{LR}(\boldsymbol{\lambda})$ non migliora [web:55][web:59]. Il numero massimo di iterazioni è fissato a 5000 e il metodo viene interrotto anticipatamente se, in un intervallo di 50 iterazioni, il valore di $z_{LR}(\boldsymbol{\lambda})$ non si riduce di almeno lo 0.01%.

La procedura complessiva, denominata *LagrangianHeuristic*, è riassunta in Figura 5.4. A ogni iterazione del sottogradiente si risolve LR per un dato vettore di

penalità λ , ottenendo il valore $z_{LR}(\lambda)$ e aggiornando il miglior upper bound z_{LR}^* . Successivamente, si calcola una soluzione euristica $\mathbf{x}'$ tramite *QuickGreedyHeuristic*, utilizzando le utilità penalizzate definite in (5.14), e la si migliora mediante *ExchangeHeuristic*; il valore z' della soluzione così ottenuta sostituisce il miglior valore corrente z^* se $z' > z^*$. Inoltre, se la condizione $\gamma z^* \leq z'$ è soddisfatta, si genera una seconda soluzione euristica $\mathbf{x}''$ con la più costosa *GreedyHeuristic*, sempre basata sulle utilità penalizzate, e il corrispondente valore z'' viene confrontato con z^* per un eventuale aggiornamento.

Listing 5.4: Algorithm LagrangianHeuristic

```

1 Set lambda = 0
2 Set z_best = -INF
3 Set z_LR_best = +INF
4
5 REPEAT
6     Compute z_LR(lambda) by solving the LagrangianProblemLR
7     IF z_LR(lambda) < z_LR_best THEN
8         z_LR_best = z_LR(lambda)
9     ENDIF
10    Compute a heuristic solution x_prime with
        QuickGreedyHeuristic
11        using penalized utilities
12    Improve x_prime with ExchangeHeuristic
13    Let z_prime = value of x_prime
14    IF gamma * z_best <= z_prime THEN
15        Compute a heuristic solution x_second with
            GreedyHeuristic
16        using penalized utilities
17        Let z_second = value of x_second
18
19        IF z_best < z_second THEN
20            z_best = z_second
21            x_best = x_second
22        ENDIF
23    ENDIF
24    IF z_best < z_prime THEN
25        z_best = z_prime
26        x_best = x_prime
27    ENDIF
28    Update penalties lambda
29
30 UNTIL subgradient stopping conditions are satisfied

```

Il parametro γ controlla la frequenza di invocazione di *GreedyHeuristic*: ponendo $\gamma = 1$ si esegue questa procedura solo quando $\mathbf{x}'$ rappresenta la migliore soluzione trovata finora, mentre per $\gamma < 1$ *GreedyHeuristic* viene attivata anche quando z' è “sufficientemente vicino” a z^* , cioè entro una distanza percentuale $100(1 - \gamma)$. Al termine di *LagrangianHeuristic* si dispone di una soluzione fattibile di valore z^* e di un upper bound z_{LR}^* derivante dal rilassamento lagrangiano, che consente di stimare la distanza massima della soluzione euristica dall’ottimo del problema originale.

5.3 Aggiornamenti ai metodi di cutting plane

I metodi di *cutting plane* costituiscono una tecnica fondamentale nella programmazione lineare intera mista (MILP), utilizzata per rafforzare iterativamente il rilassamento lineare continuo mediante l’aggiunta di disuguaglianze valide (*tagli*) che eliminano soluzioni frazionarie non ammissibili per il problema discreto. Nel contesto del problema di assegnazione sprint-user story, i tagli di precedenza OR/AND garantiscono che, per eseguire una user story j nello sprint i , almeno una (OR) oppure tutte (AND) le sue dipendenze predecessori siano state completate negli sprint precedenti ($i_1 \leq i$).

Durante la fase di sperimentazione ci si è resi conto di alcune possibili limitazioni presenti nella componente di branch-and-cut dell’euristica matheuristic proposta da [BGRT14]. In particolare, l’implementazione originale della funzione `CuttingPlaneHeu` presentava: assenza di controllo sulla profondità dell’albero di branch-and-bound e mancanza di meccanismi di deduplicazione.

La nuova implementazione `CuttingPlaneHeu_new` introduce i seguenti potenziamenti:

- **Limitazione alla profondità dell’albero:** i tagli vengono generati esclusivamente per nodi con $\text{depth} \leq 3$, concentrando lo sforzo computazionale dove l’impatto sul bound è massimo, cercando quindi di limitare l’uso di risorse computazionali per l’esecuzione di tagli di poco conto. La adozione di $\text{depth} \leq 3$ è dovuta a un’analisi sperimentale che ha mostrato come la maggior parte dei miglioramenti significativi del bound si ottenga nei

primi livelli dell'albero di branch-and-bound e che la profondità media fosse $\text{Avg_depth} = 2.7$.

- **Generazione multi-cut:** ogni callback può richiamare la funzione che aggiunge i tagli fin a 10 volte, accelerando la scoperta dei tagli da inserire.
- **Deduplicazione mediante hashing FNV-1a:** un checksum a 64 bit sulle colonne non nulle previene la rigenerazione di tagli duplicati.
- **Tolleranza di violazione ridotta:** $\epsilon = 10^{-6}$ per una maggiore selettività.

Listing 5.5: Algoritmo multi-cut per la callback di generazione tagli

```
1 myHeucutcallbackNew(env, cbdata, wherefrom, cbhandle):
2     if FlagLP or depth > 3:
3         return
4
5     Get node solution x[0..numcols-1]
6
7     numCutsAdded = 0
8     FOR iter = 1 TO 10:
9         if NOT OR_AND_InequalitiesNew(cutinfo):
10             break
11
12         Add cut with CPX_USECUT_FILTER
13         numCutsAdded++
14
15     if numCutsAdded > 0:
16         useraction_p = CPX_CALLBACK_SET
```

La funzione `OR_AND_InequalitiesNew` implementa correttamente la generazione dei tagli di precedenza considerando la dimensione temporale:

Listing 5.6: Generazione corretta dei tagli di precedenza OR-AND

```
1 OR_AND_InequalitiesNew(cutinfo):
2     FOR j IN user_stories:
3         FOR i IN sprints:
4             if x[i*n+j] < 1e-9:
5                 continue
6
7             // Precedenza OR: almeno UNA dipendenza
8             completata
9             lhs = sum_{i1<=i, u in UOR[j]} x[i1*n+u]
10            if lhs - x[i*n+j] < -1e-6:
11                Build cut: sum_{i1<=i, u in UOR[j]} x[i1*n+u]
12                ] >= x[i*n+j]
13                if checksum != last_cut_checksum:
14                    return VIOLATED
15
16            // Precedenza AND: TUTTE le dipendenze
17            completate
18            lhs = sum_{i1<=i, u in UAND[j]} x[i1*n+u]
19            if lhs - |UAND[j]|*x[i*n+j] < -1e-6:
20                Build cut: sum_{i1<=i, u in UAND[j]} x[i1*n+u]
21                ] >= |UAND[j]|*x[i*n+j]
22                if checksum != last_cut_checksum:
23                    return VIOLATED
24
25            return NO_VIOLATION
```

5.4 Funzioni obiettivo

Per valutare approcci alternativi alla funzione obiettivo originale (4.1), sono state introdotte cinque nuove funzioni obiettivo, progettate per catturare diversi aspetti critici della pianificazione degli sprint. La motivazione principale di questa esten-

sione risiede nella natura multi-obiettivo intrinseca del problema di sprint planning: oltre alla massimizzazione del valore di business, i team di sviluppo devono simultaneamente gestire l'incertezza nelle stime, rispettare le dipendenze tecniche, e ottimizzare l'utilizzo della capacità disponibile. Le funzioni proposte permettono di esplorare sistematicamente come diverse priorità strategiche influenzino la qualità e la fattibilità delle soluzioni ottenute.

5.4.1 Formulazioni matematiche

Le nuove funzioni obiettivo sono definite come segue:

$$z_P^{cr} = \max \sum_{k=1}^m \sum_{i=1}^k \sum_{j=1}^n u_j (r_j^{cr} x_{ij} + a_j y_{ij}) dp_{ij} + ub_{ij} \quad (5.23)$$

$$z_P^{unc} = \max \sum_{k=1}^m \sum_{i=1}^k \sum_{j=1}^n u_j (r_j^{unc} x_{ij} + a_j y_{ij}) \quad (5.24)$$

$$z_P^{unc+mod} = \max \sum_{k=1}^m \sum_{i=1}^k \sum_{j=1}^n u_j (r_j^{unc} x_{ij} + a_j y_{ij}) dp_{ij} + ub_{ij} \quad (5.25)$$

$$z_P^{load} = \max \sum_{k=1}^m \sum_{i=1}^k \sum_{j=1}^n p_j (r_j^{unc} x_{ij} + a_j y_{ij}) \quad (5.26)$$

$$z_P^{load+mod} = \max \sum_{k=1}^m \sum_{i=1}^k \sum_{j=1}^n p_j (r_j^{unc} x_{ij} + a_j y_{ij}) dp_{ij} + ub_{ij} \quad (5.27)$$

I nuovi parametri introdotti sono:

- $dp_{ij} \in [0.25, 1.0]$: *development penalty*, coefficiente che quantifica il grado di completamento delle dipendenze della user story j prima dello sprint i . Assume valore 1.0 se tutte le dipendenze predecessori sono state completate, e decade progressivamente fino a 0.25 se nessuna dipendenza è stata ancora eseguita.
- $ub_{ij} \geq 0$: *urgency bonus*, bonus proporzionale al numero di user story che hanno j come dipendenza e sono pianificate negli sprint successivi. Vale 0 se j non ha dipendenze in uscita.

5.4.2 Obiettivi delle diverse funzioni

Ciascuna funzione obiettivo persegue una specifica strategia di pianificazione:

Funzione z_P^{cr} (Criticità con modificatori) (5.23). Rappresenta la formulazione originale del problema, che massimizza il valore di business ponderato per la criticità r_j^{cr} delle user story. I modificatori dp_{ij} e ub_{ij} penalizzano l'assegnazione precoce di story con dipendenze non soddisfatte e premiano le story “critiche” che sbloccano molte altre attività. **Obiettivo:** bilanciare valore di business e rispetto delle dipendenze architetturali.

Funzione z_P^{unc} (Incertezza senza modificatori) (5.24). Incorpora il parametro r_j^{unc} , che quantifica l'incertezza nelle stime di sviluppo della user story j (tipicamente derivata dalla deviazione standard delle stime del team o dalla volatilità storica di story simili). Questa funzione privilegia l'assegnazione precoce delle user story più incerte, garantendo buffer temporale sufficiente per gestire eventuali ritardi o scoperte impreviste durante l'implementazione. **Obiettivo:** riduzione del rischio di schedule slippage concentrando le attività ad alto rischio nei primi sprint.

Funzione $z_P^{unc+mod}$ (Incertezza con modificatori) (5.25). Estende (5.24) integrando i modificatori dp_{ij} e ub_{ij} per combinare la gestione dell'incertezza con il rispetto delle dipendenze. **Obiettivo:** bilanciare il rischio di schedule slippage e il rispetto delle dipendenze.

Funzione z_P^{load} (Carico di lavoro senza modificatori) (5.26). Utilizza il peso p_j (espresso in story points, giorni-persona, o altra metrica di effort) per massimizzare il carico di lavoro complessivo assegnato agli sprint. Questa funzione mira a saturare la capacità disponibile, riducendo gli sprechi di risorse e accelerando il completamento del progetto. **Obiettivo:** massimizzazione del throughput e riduzione della durata totale del progetto.

Funzione $z_P^{load+mod}$ (Carico di lavoro con modificatori) (5.27). Combina l'ottimizzazione del carico con i modificatori per evitare che la saturazione degli

sprint violi le dipendenze architetturali. **Obiettivo:** throughput elevato mantenendo la fattibilità tecnica della pianificazione.

5.4.3 Ruolo dei parametri modificatori

La presenza o assenza dei modificatori dp_{ij} e ub_{ij} definisce due varianti per ciascuna strategia:

Varianti senza modificatori ((5.24), (5.26)). Rappresentano formulazioni “pure” che ottimizzano esclusivamente il criterio principale (incertezza o carico) senza considerazioni esplicite sulle dipendenze. Queste varianti sono utili come baseline per valutare l’impatto dei modificatori, ma possono generare soluzioni che violano frequentemente i vincoli di precedenza, richiedendo correzioni successive.

Varianti con modificatori ((5.23), (5.25), (5.27)). Integrano due meccanismi di guida della ricerca:

1. **Development penalty** (dp_{ij}): penalizza l’assegnazione di user story j allo sprint i se le sue dipendenze predecessori non sono state completate, incentivando l’algoritmo a rispettare naturalmente l’ordinamento topologico del grafo delle dipendenze. Questo riduce drasticamente il numero di soluzioni intermedie infeasible esplorate durante la ricerca.
2. **Urgency bonus** (ub_{ij}): premia l’esecuzione anticipata delle user story “radice” (senza dipendenze in ingresso) o delle story che sbloccano molte attività successive. Questo favorisce pianificazioni che costruiscono progressivamente l’infrastruttura necessaria per le feature dipendenti, evitando colli di bottiglia negli sprint finali.

5.4.4 Confronto sperimentale

I risultati computazionali di confronto sono riportati nella Sezione 6.5.

Chapter 6

Risultati ottenuti

6.1 dati utilizzati

Gli algoritmi considerati in questo lavoro sono stati implementati in C++ all'interno di Microsoft Visual Studio 2024.

Per la valutazione sperimentale sono stati utilizzati sia progetti reali sia progetti sintetici generati artificialmente.

I progetti reali provengono da contesti applicativi differenti e sono stati realizzati da aziende italiane che adottano metodologie Agile da diversi anni. Di seguito se ne riassumono le principali caratteristiche.

- **PayTV** Progetto di sviluppo di un data mart per una grande azienda del settore pay TV. Caratteristiche principali:
 - durata complessiva: 8 mesi;
 - numero di user stories: 44;
 - numero di sprint: 10;
 - durata media degli sprint: 17 giorni;
 - numero di dipendenze: 35 (prevalentemente di tipo AND);
 - numero di affinità: 1;
 - velocità di sviluppo: 2,43 story point/giorno, stimata empiricamente su dati storici.

- **Web** Progetto di sviluppo di un sito web complesso basato su un Content Management System. Caratteristiche principali:
 - numero di user stories: 104 (maggiore di PayTV);
 - numero di sprint: 4;
 - durata degli sprint: 10 giorni ciascuno;
 - durata complessiva: 40 giorni;
 - velocità di sviluppo: 6 story point/giorno;
 - dipendenze: 6 dipendenze in catena;
 - nessun vincolo di affinità.

Dal punto di vista dell'efficacia, tutti i piani ottimali generati dagli algoritmi sono stati giudicati fattibili e realistici dai project manager; nella maggior parte dei casi, le differenze rispetto ai piani originariamente proposti dai team sono state valutate come migliorative in termini di composizione degli sprint (si veda [?] per un'analisi dettagliata).

Per mettere maggiormente sotto stress gli algoritmi, è stato inoltre generato un insieme di progetti sintetici. Il generatore di istanze opera come segue:

1. crea le user stories assegnando in modo casuale utilità, rischio e complessità;
2. aggiunge gruppi di dipendenze, organizzati sia come catene (una story dipende da una sola story) sia come DAG (una story dipende da più stories);
3. definisce insiemi di storie affini;
4. fissa la capacità di sprint a 45 story point e la velocità di sviluppo a 3 story point/giorno (ogni sprint dura quindi 15 giorni).

6.1.1 Raggruppamento dei progetti

La Tabella 6.1 riassume le caratteristiche principali di ciascun progetto, riportando:

- il numero n di user stories;
- il numero massimo m di sprint;

- il numero n_{aff} di stories coinvolte in almeno un vincolo di affinità;
- la cardinalità degli insiemi U^{OR} e U^{AND} ;
- la lunghezza massima l_{max} dei gruppi di dipendenze;
- il numero massimo d_{max} di dipendenze che coinvolgono una singola story.

I progetti sono stati raggruppati in cinque cluster:

Gruppo A Contiene i progetti reali (PayTV e Web).

- Parametri derivati da casi industriali.
- Mix realistico di dimensione, dipendenze e (eventuale) presenza di affinità.

Gruppi B e C Raccolgono progetti sintetici che combinano in modo vario i parametri sopra descritti.

- Dimensioni crescenti in termini di numero di stories e sprint.
- Diversa distribuzione di dipendenze OR/AND e lunghezza delle catene/DAG.
- Presenza o assenza di affinità secondo configurazioni eterogenee.

Gruppo D Progetti sintetici in cui l'utilità è fortemente correlata alla complessità.

- Per ogni story, la complessità è sempre pari al doppio dell'utilità.
- Scenario utile per analizzare casi in cui “alto valore” implica automaticamente “alta complessità”.

Gruppo E Progetti caratterizzati da user stories ad elevata complessità.

- Ogni sprint può contenere al più 5 stories.
- Configurazione concepita per studiare il comportamento degli algoritmi in presenza di capacità molto stringenti.

6.1. DATI UTILIZZATI

Table 6.1: Problem instances

Group	Proj. Name	n	m	n _{aff}	U ^{OR}	U ^{AND}	l _{max}	d _{max}
A – Real								
PayTV		44	12	2	8	27	6	5
Web		104	6	5	0	4	4	1
B – Basic (Set 1)								
25Chain-1		25	9	0	0	12	4	1
25Graph-1		25	8	0	10	1	2	2
25Affinity-1		25	9	6	5	5	2	2
50Chain-1		50	12	0	0	20	4	1
50Graph-1		50	11	0	8	10	2	2
50Affinity-1		50	12	6	8	9	2	2
75Chain-1		75	17	0	0	35	5	1
75Graph-1		75	19	0	13	20	2	2
75Affinity-1		75	17	6	17	13	2	3
100Chain-1		100	20	0	0	40	5	1
100Graph-1		100	23	0	20	14	2	3
100Affinity-1		100	22	6	16	14	3	4
C – Basic (Set 2)								
25Chain-2		25	8	0	0	12	2	1
25Graph-2		25	8	0	3	8	3	2
25Affinity-2		25	9	6	7	4	3	2
50Chain-2		50	13	0	0	10	5	1
50Graph-2		50	13	0	13	8	4	5
50Affinity-2		50	13	6	13	9	3	3
75Chain-2		75	17	0	0	36	3	1
75Graph-2		75	18	0	14	16	2	2
75Affinity-2		75	18	6	17	13	2	2
100Chain-2		100	22	0	0	30	5	1
100Graph-2		100	22	0	12	15	5	7
100Affinity-2		100	22	6	11	14	3	2
D – Correlated								
25Chain-3		25	15	0	0	12	4	1
25Graph-3		25	15	0	5	7	5	4
25Affinity-3		25	13	6	5	4	2	2
50Chain-3		50	22	0	0	20	4	1
50Graph-3		50	22	0	7	9	2	2
50Affinity-3		50	21	6	9	6	2	2
75Chain-3		75	31	0	0	36	6	1
75Graph-3		75	29	0	12	17	2	2
75Affinity-3		75	33	6	20	7	2	2
100Chain-3		100	38	0	0	40	8	1
100Graph-3		100	40	0	16	14	3	3
100Affinity-3		100	43	6	16	13	2	2

Table 6.2: Problem instances

Group	Proj. Name	n	m	n _{aff}	U ^{OR}	U ^{AND}	l _{max}	d _{max}
B – Few								
25Chain-4		25	12	0	5	7	4	1
25Graph-4		25	13	0	5	6	5	4
25Affinity-4		25	14	6	3	7	2	2
50Chain-4		50	21	0	9	11	4	1
50Graph-4		50	21	0	2	13	2	2
50Affinity-4		50	21	6	6	8	2	2
75Chain-4		75	27	0	15	21	6	1
75Graph-4		75	29	0	12	12	2	4
75Affinity-4		75	29	6	10	12	2	2
100Chain-4		100	40	0	24	21	9	1
100Graph-4		100	40	0	8	17	2	2
100Affinity-4		100	39	6	18	10	2	2

6.1.2 Generazione di nuove istanze di test

I dataset utilizzati per la valutazione sperimentale degli algoritmi proposti in [BGRT14] presentano alcune limitazioni significative: in particolare, mancano completamente i parametri relativi al rischio (r_j^{cr}) e all'incertezza (r_j^{unc}) delle user story, essendo stati concepiti prima dell'introduzione di queste metriche nella modellazione del problema. Questa carenza rende impossibile una valutazione empirica significativa delle nuove funzioni obiettivo introdotte nella Sezione 5.4, che si basano proprio su questi parametri per guidare le decisioni di pianificazione.

Per ovviare a tale limitazione, è stato sviluppato un generatore sintetico di istanze che permette il controllo completo sui seguenti parametri configurabili:

- numero di user story (n) e numero di sprint (m);
- range di utilità delle user story: $u_j \in [u_{\min}, u_{\max}]$;
- range di rischio (criticità): $r_j^{cr} \in [r_{\min}^{cr}, r_{\max}^{cr}]$;
- range di incertezza: $r_j^{unc} \in [r_{\min}^{unc}, r_{\max}^{unc}]$;
- range di story point (peso): $p_j \in [p_{\min}, p_{\max}]$;
- range di capacità degli sprint: $c_i \in [c_{\min}, c_{\max}]$;

- probabilità di dipendenza (ρ_{dep}): probabilità che una user story presenti vincoli di precedenza;
- numero massimo di dipendenze per user story (d_{max}).

Algoritmo di generazione. Il generatore opera in due fasi sequenziali. Nella prima fase, vengono campionati casualmente da distribuzioni uniformi i parametri numerici di ciascuna user story (u_j , r_j^{cr} , r_j^{unc} , p_j) e la capacità di ogni sprint (c_i). Nella seconda fase, viene costruito il grafo delle dipendenze: per ogni user story j , con probabilità ρ_{dep} vengono estratti casualmente fino a d_{max} predecessori dall'insieme delle user story già generate, verificando l'assenza di cicli mediante visita topologica. Questo garantisce che il grafo di dipendenza risultante sia un DAG (Directed Acyclic Graph), condizione necessaria per la fattibilità del problema.

6.1.3 Benchmark suite generato

Utilizzando il generatore descritto, sono state create 25 istanze sintetiche organizzate in 5 gruppi tematici, ciascuno composto da 5 progetti con caratteristiche dimensionali e strutturali omogenee:

Gruppo 1 (istanze small). Progetti con 25 user story distribuite su 9 sprint, con al massimo 3 dipendenze per story. Queste istanze rappresentano team Scrum di piccole dimensioni che lavorano su progetti di breve durata (circa 2-3 mesi).

Gruppo 2 (istanze medium). Progetti con 60 user story distribuite su 30 sprint, con al massimo 3 dipendenze per story. Simulano progetti di media complessità con cicli di sviluppo di 6-12 mesi.

Gruppo 3 (istanze large). Progetti con 120 user story distribuite su 30 sprint, con al massimo 2 dipendenze per story ma densità di story per sprint più elevata. Rappresentano progetti complessi con backlog ampio ma orizzonte temporale contenuto.

Gruppo 4 (istanze extra-large). Progetti con 300 user story distribuite su 110 sprint, con al massimo 2 dipendenze per story. Queste istanze modellano progetti enterprise di lunga durata (oltre 2 anni) con grafi di dipendenza molto ampi.

Gruppo 5 (istanze Fibonacci). Progetti con 60 user story distribuite su 25 sprint, con al massimo 3 dipendenze per story. La caratteristica distintiva di questo gruppo è l'uso della sequenza di Fibonacci (1, 2, 3, 5, 8, 13, 21) per gli story point, in conformità alle linee guida ufficiali dello Scrum framework [?], che raccomandano l'uso di scale non lineari per riflettere l'incertezza crescente nelle stime di effort elevato.

Distribuzione degli story point. Nei gruppi 1–4, gli story point sono espressi come giorni-uomo e seguono una distribuzione uniforme nell'intervallo $[p_{\min}, p_{\max}]$, modellando una situazione realistica in cui le user story hanno complessità variabile. Nel gruppo 5, gli story point sono campionati uniformemente dall'insieme discreto $\{1, 2, 3, 5, 8, 13, 21\}$, rispecchiando la pratica industriale di Planning Poker e della stima relativa. Questa distinzione permette di valutare la robustezza degli algoritmi rispetto a diverse convenzioni di stima adottate dai team di sviluppo.

6.1.4 Caratteristiche strutturali delle istanze

La Tabella 6.3 riporta le principali metriche strutturali delle 25 istanze generate, dove:

- n : numero totale di user story;
- m : numero di sprint pianificati;
- n_{aff} : numero di user story effettivamente assegnate nella soluzione ottima;
- $|U^{OR}|$: numero totale di vincoli di precedenza OR nel grafo;
- $|U^{AND}|$: numero totale di vincoli di precedenza AND nel grafo;
- $l_{\max}$: lunghezza massima di una catena di dipendenze (profondità del DAG);
- $d_{\max}$: grado in-degree massimo nel grafo delle dipendenze.

6.1. DATI UTILIZZATI

Table 6.3: Instances generated with new data generator

Group	Proj. Name	n	m	n_{aff}	$ U^{OR} $	$ U^{AND} $	l_{max}	d_{max}
Gruppo 1								
instance_06_01		25	9	11	2	3	3	3
instance_06_02		25	9	11	3	5	3	4
instance_06_03		25	9	12	2	6	3	5
instance_06_04		25	9	13	3	3	3	3
instance_06_05		25	9	13	3	5	3	4
Gruppo 2								
instance_06_01		60	30	14	9	9	3	6
instance_06_02		60	30	14	6	10	3	6
instance_06_03		60	30	14	7	8	3	4
instance_06_04		60	30	11	5	7	3	5
instance_06_05		60	30	17	5	7	3	3
Gruppo 3								
instance_06_01		120	30	14	18	12	2	3
instance_06_02		120	30	17	12	19	2	5
instance_06_03		120	30	16	13	22	2	3
instance_06_04		120	30	14	16	25	2	5
instance_06_05		120	30	12	21	18	2	4
Gruppo 4								
instance_06_01		300	110	16	41	42	2	6
instance_06_02		300	110	17	48	52	2	4
instance_06_03		300	110	16	50	46	2	5
instance_06_04		300	110	17	42	58	2	5
instance_06_05		300	110	12	40	52	2	5
Gruppo 5								
instance_06_01		60	25	14	10	12	3	5
instance_06_02		60	25	16	9	12	3	6
instance_06_03		60	25	12	5	11	3	3
instance_06_04		60	25	14	10	4	3	4
instance_06_05		60	25	14	15	6	3	4

Le istanze mostrano una varietà significativa sia in termini dimensionali (da 25 a 300 user story) sia in termini di complessità del grafo delle dipendenze (profondità massima delle catene compresa tra 2 e 3, grado massimo in-degree tra 3 e 6). Questa eterogeneità permette di testare la scalabilità e la robustezza degli algoritmi proposti su un ampio spettro di scenari operativi realistici.

6.2 KPI di valutazione

Per valutare in modo sistematico le prestazioni degli algoritmi e l'efficacia delle funzioni obiettivo proposte, sono stati adottati due insiemi complementari di indicatori: i KPI computazionali, che misurano l'efficienza dell'ottimizzazione, e i KPI di qualità della pianificazione, che valutano l'idoneità pratica delle soluzioni nel contesto Agile.

6.2.1 KPI computazionali

I seguenti indicatori, già utilizzati in [BGRT14], permettono di confrontare le prestazioni algoritmiche e la convergenza verso l'ottimalità:

Z_{OPT} Valore della funzione obiettivo nella soluzione ottima certificata da IBM ILOG CPLEX. Costituisce il benchmark di riferimento per valutare la qualità delle soluzioni euristiche.

Z_{HEU} Valore della funzione obiettivo nella migliore soluzione euristica individuata dall'algoritmo matheuristic entro il limite temporale imposto.

Gap (%) Gap di ottimalità percentuale tra la migliore soluzione euristica e il bound superiore del miglior nodo rimanente nell'albero di ricerca:

$$\text{Gap} = 100 \times \frac{Z_{\text{OPT}}^{\text{ub}} - Z_{\text{HEU}}}{Z_{\text{OPT}}^{\text{ub}}}$$

dove $Z_{\text{OPT}}^{\text{ub}}$ rappresenta il miglior bound superiore disponibile. Valori bassi indicano che la soluzione euristica è prossima all'ottimo certificabile.

Nodes Numero totale di nodi esplorati nell'albero di branch-and-bound durante la ricerca. Quantifica lo sforzo computazionale necessario: meno nodi implicano una ricerca più efficiente grazie a bound più stretti o euristiche di branching più intelligenti.

NCuts Numero complessivo di tagli (cutting planes) generati dalle callback durante l'ottimizzazione, includendo:

- vincoli di precedenza OR/AND (4.4)–(4.5) inseriti come lazy constraints;
- disuguaglianze di dominanza (DIs) che eliminano soluzioni dominate;
- logic-based cutting inequalities (LCIs) che rafforzano il rilassamento lineare sfruttando la struttura logica del problema.

Un numero elevato di tagli può indicare sia efficacia nel rafforzamento del rilassamento, sia presenza di molte violazioni nelle soluzioni frazionarie.

GHeu (%) Gap percentuale di qualità della soluzione euristica rispetto alla soluzione ottima nota:

$$G_{\text{heu}} = 100 \times \frac{Z_{\text{OPT}} - Z_{\text{HEU}}}{Z_{\text{OPT}}}$$

Misura direttamente la perdita di qualità dovuta all'uso dell'euristica anziché dell'ottimizzazione esatta. Valori prossimi allo zero indicano che l'euristica trova soluzioni quasi ottimali.

Time (s) Tempo computazionale totale espresso in secondi, dall'inizializzazione dell'algoritmo fino al termine dell'esecuzione (sia per raggiungimento dell'ottimo certificato sia per time limit). Riflette la scalabilità pratica dell'approccio.

6.2.2 KPI di qualità della pianificazione

Per valutare l'idoneità delle soluzioni nel contesto della pianificazione Agile reale, sono stati introdotti nuovi indicatori che misurano aspetti strategici e operativi non catturati dalla sola funzione obiettivo:

Efficienza temporale e utilizzo delle risorse.

N.S.O. *Numero Sprint Occupati*: conta gli sprint effettivamente utilizzati nella soluzione (quelli con almeno una user story assegnata). Valori bassi indicano consegna rapida del progetto, concentrando il lavoro in meno iterazioni e riducendo l'overhead di gestione. Un numero elevato può segnalare frammentazione eccessiva o sottoutilizzo della capacità.

A.S.U. (%) *Media Utilizzo Sprint*: percentuale media di capacità c_i effettivamente sfruttata negli sprint occupati:

$$\text{A.S.U.} = \frac{100}{|\{i : \exists j, x_{ij} = 1\}|} \sum_{i: \exists j, x_{ij}=1} \frac{\sum_j p_j x_{ij}}{c_i}$$

Valori elevati (prossimi al 100%) indicano saturazione efficiente della capacità disponibile, minimizzando gli sprechi. Valori troppo bassi suggeriscono che la pianificazione lascia risorse inutilizzate.

Distribuzione del rischio e dell'incertezza.

D.R. *Deviazione Standard del Rischio*: quantifica la variabilità del rischio medio aggregato $\bar{r}_i^{cr} = \frac{1}{|\{j: x_{ij}=1\}|} \sum_j r_j^{cr} x_{ij}$ tra gli sprint occupati:

$$\text{D.R.} = \sqrt{\frac{1}{|\{i : \exists j, x_{ij} = 1\}|} \sum_{i: \exists j, x_{ij}=1} (\bar{r}_i^{cr} - \bar{r}^{cr})^2}$$

dove $\bar{r}^{cr}$ è il rischio medio globale. Bassa deviazione indica distribuzione equilibrata del rischio tra gli sprint, evitando concentrazioni pericolose o sprint eccessivamente sbilanciati.

D.U. *Deviazione Standard dell'Incertezza*: analoga a D.R., misura la variabilità dell'incertezza media $\bar{r}_i^{unc}$ tra gli sprint. Bassa variabilità favorisce gestione prevedibile e uniforme delle attività ad alta incertezza, evitando sprint particolarmente volatili.

R.MIN *Rischio Minimo*: identifica il valore minimo di $\bar{r}_i^{cr}$ tra tutti gli sprint occupati. Valori troppo bassi possono segnalare sprint composti esclusivamente da user story “sicure”, indicando potenziale sottoutilizzo delle capacità del team di gestire attività complesse.

R.MAX *Rischio Massimo*: identifica il valore massimo di $\bar{r}_i^{cr}$ tra tutti gli sprint occupati. Valori eccessivamente elevati segnalano sprint critici con esposizione al rischio potenzialmente ingestibile, che richiedono attenzione particolare dal team e dagli stakeholder.

U.MIN *Incertezza Minima*: valore minimo di $\bar{r}_i^{unc}$ tra gli sprint occupati. Garantisce che ogni sprint contenga user story con un livello minimo di incertezza, evitando sprint composti solo da attività banali o completamente comprese.

U.MAX *Incertezza Massima*: valore massimo di $\bar{r}_i^{unc}$ tra gli sprint occupati. Identifica l'esposizione all'incertezza nel caso peggiore, essenziale per valutare la robustezza della pianificazione rispetto a imprevisti e variazioni nelle stime.

Progressione del valore consegnato.

H.U.S. *Half Utility Sprint*: indice dello sprint in cui viene raggiunto almeno il 50% dell'utilità totale cumulata:

$$\text{H.U.S.} = \min \left\{ k : \sum_{i=1}^k \sum_{j=1}^n u_j x_{ij} \geq \frac{1}{2} \sum_{j=1}^n u_j \right\}$$

Questo indicatore misura il *time-to-half-value*, ovvero la rapidità con cui il progetto inizia a consegnare valore significativo agli stakeholder. Valori bassi (sprint iniziali) indicano pianificazioni che prioritizzano il rilascio anticipato di feature ad alto valore, conformemente ai principi Agile di early value delivery.

6.2.3 Interpretazione congiunta dei KPI

I KPI computazionali e i KPI di qualità della pianificazione forniscono prospettive complementari: i primi valutano l'efficienza matematica dell'ottimizzazione, mentre i secondi misurano la traducibilità pratica delle soluzioni in piani di sprint realistici e bilanciati. Una soluzione ideale dovrebbe eccellere in entrambe le dimensioni, presentando gap di ottimalità ridotti (qualità computazionale) e distribuzione equilibrata di rischio, incertezza e valore (qualità gestionale). L'analisi comparativa delle funzioni obiettivo nella Sezione 6.5 utilizza sistematicamente questi indicatori per identificare i trade-off tra le diverse strategie di pianificazione.

6.3 risultati computazionali

Nella Tabella 6.4 sono riportati i risultati ottenuti dall'esecuzione degli algoritmi proposti da [BGRT14] con l'aggiornamento del codice per supportare l'attuale versione della libreria CPLEX.

Nei test computazionali è stato fissato un limite di tempo di 600 secondi per tutti i risultati riportati di seguito, per cui quando IBM Ilog Cplex non trova una soluzione fattibile per un'istanza entro il limite di tempo assegnato, riportiamo il carattere “–” nelle colonne z , Gap e $BGap$.

6.3. RISULTATI COMPUTAZIONALI

Table 6.4: Results obtained solving the basic model with IBM Ilog Cplex and adding valid inequalities

Istanza	CPLEX					Heuristic			Lagrangian Heuristic		
	Zopt	Gap	Time	Zheu	Time	GHeu	Zheu	Time	GHeu		
paytv-2	97746.0	0.00	0	22	0.07	98873.0	0.03	-1.15	98873.0	17.47	-1.15
web-2	32745.5	0.00	0	1	0.11	32450.9	0.01	0.90	32683.6	44.83	0.19
25Chain-1	17000.6	0.00	0	2	0.01	17075.0	0.02	-0.44	17075.0	0.23	-0.44
25Graph-1	13515.1	0.01	722	10	0.11	13583.7	0.01	-0.51	13583.7	0.07	-0.51
25Affinity-1	18567.3	0.00	686	18	0.21	18298.0	0.01	1.45	18505.1	0.34	0.34
50Chain-1	42449.3	0.00	0	6	0.02	42689.1	0.02	-0.56	42689.1	1.96	-0.56
50Graph-1	34802.7	0.00	0	16	0.04	35346.8	0.01	-1.56	35346.8	0.73	-1.56
50Affinity-1	40545.1	0.01	2842	33	1.20	40623.2	0.03	-0.19	40623.2	3.06	-0.19
75Chain-1	91864.8	0.00	0	53	0.20	93223.5	0.03	-1.48	93223.5	25.93	-1.48
75Graph-1	93196.4	0.01	2483	95	1.36	94302.5	0.03	-1.19	94302.5	30.44	-1.19
75Affinity-1	76076.2	0.01	106389	83	69.03	76525.3	0.09	-0.59	76525.3	16.39	-0.59
100Chain-1	137043.9	0.01	292948	283	186.82	142276.9	0.09	-3.82	142276.9	59.41	-3.82
100Graph-1	149530.4	0.22	1522000	169	600.47	151302.6	0.09	-1.18	151302.6	64.07	-1.18
100Affinity-1	136175.4	0.12	459400	136	600.17	136426.7	0.24	-0.18	136426.7	78.29	-0.18
25Chain-2	13384.9	0.00	0	4	0.04	13346.6	0.01	0.29	13346.6	0.33	0.29
25Graph-2	17574.2	0.00	0	3	0.04	17596.1	0.03	-0.12	17596.1	0.19	-0.12
25Affinity-2	13371.9	0.00	0	2	0.03	13445.1	0.01	-0.55	13445.1	0.44	-0.55
50Chain-2	46761.2	0.01	89	39	0.19	46986.5	0.03	-0.48	46986.5	1.68	-0.48
50Graph-2	38844.8	0.00	0	26	0.12	39487.7	0.04	-1.65	39487.7	4.68	-1.65
50Affinity-2	46153.1	0.01	21027	110	13.34	47976.3	0.04	-3.95	47976.3	3.71	-3.95
75Chain-2	83638.0	0.00	0	41	0.26	84754.3	0.04	-1.33	84754.3	15.13	-1.33
75Graph-2	74014.5	0.00	0	6	0.08	74264.7	0.06	-0.34	74264.7	26.76	-0.34
75Affinity-2	84349.9	0.06	545900	133	600.16	85075.3	0.06	-0.86	85075.3	38.80	-0.86
100Chain-2	134393.0	0.32	1276186	428	600.52	136437.4	0.13	-1.52	136437.4	25.60	-1.52
100Graph-2	134930.5	0.33	1231297	151	600.46	136397.7	0.10	-1.09	136397.7	64.14	-1.09
100Affinity-2	136523.5	0.17	488700	138	600.21	137159.8	0.10	-0.47	137159.8	50.99	-0.47
25Chain-3	2855.2	0.01	14102	96	4.09	2864.0	0.07	-0.31	2864.0	0.71	-0.31
25Graph-3	1935.7	0.01	182810	80	20.05	1963.0	0.06	-1.41	1963.0	0.25	-1.41
25Affinity-3	2002.4	0.01	2710	39	2.22	1966.5	0.06	1.79	1994.9	0.46	0.37
50Chain-3	6262.1	0.84	680201	292	600.27	6300.6	0.23	-0.61	6300.6	6.74	-0.61
50Graph-3	5909.9	1.58	1969592	238	600.47	5952.4	0.15	-0.72	5952.4	4.01	-0.72
50Affinity-3	5229.6	0.68	268400	152	600.23	5210.4	0.14	0.37	5210.4	3.81	0.37
75Chain-3	12222.7	2.47	536268	912	600.42	12672.7	0.27	-3.68	12672.7	38.90	-3.68
75Graph-3	11398.2	0.86	1433100	582	600.88	11467.9	0.42	-0.61	11467.9	4.65	-0.61
75Affinity-3	14443.6	1.67	153200	294	600.14	14323.2	0.41	0.83	14436.8	5.98	0.05
100Chain-3	19530.4	1.90	335300	1270	600.51	19872.7	0.46	-1.75	19872.7	37.90	-1.75
100Graph-3	19951.9	0.80	836828	308	600.55	19834.5	0.61	0.59	19925.1	43.16	0.14
100Affinity-3	25512.2	2.44	69100	756	600.28	25737.6	0.59	-0.88	25737.6	28.25	-0.88
25Chain-4	19033.8	0.00	0	14	0.07	19183.5	0.05	-0.79	19183.5	2.02	-0.79
25Graph-4	21261.5	0.00	26	46	0.24	21784.2	0.07	-2.46	21784.2	0.90	-2.46
25Affinity-4	21216.1	0.00	0	24	0.24	20638.5	0.06	2.72	20638.5	0.30	2.72
50Chain-4	61557.3	0.01	1075	153	1.50	63368.7	0.14	-2.94	63368.7	7.97	-2.94
50Graph-4	66307.3	0.79	1833953	132	600.47	67815.8	0.14	-2.27	67815.8	5.98	-1.21
50Affinity-4	60073.0	0.03	431682	52	600.08	60005.8	0.14	0.11	60005.8	5.81	0.11
75Chain-4	110750.2	5.63	942700	779	600.86	119950.6	0.29	-8.31	119950.6	47.73	-8.31

6.4 variazione dei parametri

All'interno del progetto e' stato condotto uno studio per valutare l'impatto di alcuni parametri sul progetto. In particolare, sono stati analizzati i seguenti aspetti:

- Effetto della riduzione del numero di sprint disponibili sul valore della soluzione e sui tempi di calcolo.
- La riduzione della capacità degli sprint e il suo impatto sulla fattibilità delle soluzioni.
- L'accorpamento di sprint una volta raggiunto un certo numero di sprint e il suo effetto sulla qualità della soluzione.
- Impatto del rilassamento del vincolo OR e AND tra le user story.

6.4.1 Riduzione sprint

La riduzione degli sprint disponibili non influisce sull'esecuzione degli algoritmi proposti fintanto che il numero di sprint rimane tale da garantire la fattibilità delle soluzioni. Questo effetto nasce dal fatto che il metodo mira a riempire gli sprint in modo ottimale: l'inserimento di sprint aggiuntivi non implica necessariamente un miglioramento della soluzione, purché la fattibilità sia preservata. Tuttavia, una riduzione eccessiva del numero di sprint disponibili può compromettere la fattibilità delle soluzioni generate.

La differenza sostanziale riguarda il valore della funzione obiettivo (4.1). Come si osserva confrontando la tabella 6.4 con la tabella 6.5, tale valore diminuisce con la diminuzione del numero di sprint disponibili. Tale comportamento è attribuibile al metodo di calcolo della funzione obiettivo, che attribuisce un peso maggiore alle user story eseguite nei primi sprint rispetto a quelle eseguite negli sprint successivi, integrando la riduzione disponibile degli sprint.

6.4. VARIAZIONE DEI PARAMETRI

Table 6.5: sprint reduction results (nSprint - 2)

Istanza	CPLEX			Heuristic			Lagrangian Heuristic		
	Zopt	Gap	Time	Zheu	Time	GHeu	Zheu	Time	GHeu
paytv-2	73007.0	0.01	5.00	78111.0	0.02	-6.99	78111.0	10.79	-6.99
web-2	20583.3	0.00	0.11	20475.6	0.01	0.52	20518.3	51.91	0.32
25Chain-1	12346.3	0.00	0.02	12598.8	0.02	-2.04	12598.8	0.27	-2.04
25Graph-1	9638.7	0.01	0.09	9707.3	0.01	-0.71	9707.3	0.13	-0.71
25Affinity-1	13615.0	0.00	0.07	13508.8	0.01	0.78	13554.9	0.24	0.44
50Chain-1	34025.6	0.00	0.03	34351.9	0.02	-0.96	34351.9	2.58	-0.96
50Graph-1	27344.7	0.00	0.03	27751.2	0.01	-1.49	27751.2	1.34	-1.49
50Affinity-1	32495.1	0.01	0.45	32604.2	0.03	-0.33	32604.2	2.50	-0.33
75Chain-1	77744.8	0.00	0.23	79709.3	0.03	-2.53	79709.3	26.77	-2.53
75Graph-1	80760.3	0.28	600.40	82119.1	0.05	-1.68	82119.1	17.07	-1.68
75Affinity-1	65034.2	0.01	49.83	65497.5	0.08	-0.71	65497.5	23.06	-0.71
100Chain-1	118537.2	1.22	600.46	124595.1	0.07	-5.11	124595.1	48.82	-5.11
100Graph-1	132670.8	0.17	600.42	134346.2	0.08	-1.26	134346.2	69.24	-1.26
100Affinity-1	120213.2	0.16	600.20	120598.9	0.20	-0.32	120598.9	40.81	-0.32
25Chain-2	9614.9	0.00	0.02	9576.6	0.01	0.40	9576.6	0.38	0.40
25Graph-2	12476.6	0.00	0.02	12498.5	0.01	-0.17	12498.5	0.27	-0.17
25Affinity-2	9981.5	0.00	0.04	10054.7	0.02	-0.73	10054.7	0.44	-0.73
50Chain-2	38063.7	0.01	0.40	38349.1	0.04	-0.75	38349.1	2.43	-0.75
50Graph-2	31119.0	0.00	0.18	31960.7	0.02	-2.70	31960.7	3.32	-2.70
50Affinity-2	37443.8	0.01	0.34	39137.5	0.02	-4.52	39137.5	5.27	-4.52
75Chain-2	70791.3	0.00	0.29	72491.9	0.04	-2.40	72491.9	20.61	-2.40
75Graph-2	63776.9	0.00	0.08	64027.1	0.05	-0.39	64027.1	7.98	-0.39
75ffinity-2	72448.4	0.08	600.15	73373.1	0.06	-1.28	73373.1	12.37	-1.28
100Chain-2	118406.4	0.44	600.60	120535.0	0.11	-1.80	120535.0	37.65	-1.80
100Graph-2	119167.4	0.28	600.47	120495.3	0.08	-1.11	120495.3	36.05	-1.11
100Affinity-2	120486.1	0.20	600.19	121172.2	0.09	-0.57	121172.2	41.91	-0.57
25Chain-3	2320.8	0.01	3.40	2329.6	0.07	-0.38	2329.6	0.71	-0.38
25Graph-3	1521.8	0.01	0.58	1530.2	0.06	-0.55	1530.2	0.50	-0.55
25Affinity-3	1585.7	0.01	1.57	1562.1	0.05	1.49	1579.8	0.53	0.37
50Chain-3	5325.4	2.62	600.23	5421.0	0.16	-1.79	5421.0	2.40	-1.79
50Graph-3	5128.6	1.53	600.30	5198.2	0.16	-1.36	5198.2	1.44	-1.36
50Affinity-3	4502.1	0.76	600.20	4484.3	0.16	0.40	4484.3	1.45	0.40
75Chain-3	10764.0	4.57	600.63	11411.3	0.31	-6.01	11411.3	17.07	-6.01
75Graph-3	10185.7	1.09	600.99	10280.8	0.47	-0.93	10280.8	9.95	-0.93
75Affinity-3	13080.7	2.13	600.31	13009.9	0.34	0.54	13031.6	2.78	0.38

6.4.2 Riduzione capacità degli sprint

La riduzione della capacità degli sprint disponibili influisce significativamente sulla fattibilità delle soluzioni generate. Una diminuzione della capacità limita il numero di user story assegnabili a ciascun sprint, rendendo più arduo soddisfare i requisiti del progetto entro i vincoli imposti. Nei casi estremi, in presenza di user story di dimensioni elevate, può verificarsi l'assenza di uno sprint con capacità sufficiente per accoglierle; in tali circostanze, risulta necessario rivalutare e suddividere la user story in sotto-story più gestibili.

Come illustrato nella tabella 6.6, una riduzione del 10% della capacità degli sprint determina una diminuzione del valore della funzione obiettivo in tutte le istanze analizzate. Tale decremento riflette una minore ottimalità delle soluzioni ottenute rispetto al caso di capacità nominale, poiché l'algoritmo deve adattarsi a vincoli più restrittivi. Si osservano inoltre istanze, come 100Chain-3 e 100Affinity-3, in cui anche una riduzione del 10% compromette irreversibilmente la fattibilità del problema. Questo fenomeno sottolinea l'importanza di preservare una capacità adeguata negli sprint per garantire sia la fattibilità delle soluzioni sia la qualità complessiva del piano di progetto.

Nella tabella 6.7 sono riportati i risultati ottenuti con una riduzione del 15% della capacità degli sprint. L'impatto sulla fattibilità si rivela ancora più marcato, con un numero maggiore di istanze prive di soluzioni fattibili. Si nota inoltre che, in casi come 100Chain-1 e 100Affinity-1, CPLEX non riesce a identificare una soluzione fattibile entro il time limit, mentre gli algoritmi euristici proposti individuano soluzioni fattibili in tempi computazionali ridotti. Questo risultato evidenzia l'efficacia degli euristici nel gestire vincoli severi, fornendo soluzioni praticabili anche in scenari complessi.

6.4. VARIAZIONE DEI PARAMETRI

Table 6.6: sprint capacity reduction results (capacity - 10%)

Istanza	CPLEX			Heuristic			Lagrangian Heuristic		
	Zopt	Gap	Time	Zheu	Time	GHeu	Zheu	Time	GHeu
paytv-2	94019.0	0.00	0.16	95383.0	0.03	-1.45	95383.0	11.95	-1.45
web-2	32068.5	0.00	0.10	31825.9	0.01	0.76	32019.6	37.23	0.15
25Chain-1	16343.7	0.00	0.03	16580.8	0.02	-1.45	16580.8	0.91	-1.45
25Graph-1	13314.5	0.00	0.01	13318.9	0.01	-0.03	13318.9	0.18	-0.03
25Affinity-1	17983.9	0.01	0.30	17758.6	0.01	1.25	17978.7	0.25	0.03
50Chain-1	40156.9	0.01	28.39	41561.2	0.02	-3.50	41561.2	2.80	-3.50
50Graph-1	33739.9	0.01	0.60	34474.2	0.02	-2.18	34474.2	1.90	-2.18
50Affinity-1	39471.5	0.01	2.73	39525.0	0.02	-0.13	39525.0	2.31	-0.13
75Chain-1	85256.9	1.34	600.64	90409.9	0.09	-6.04	90409.9	31.33	-6.04
75Graph-1	90010.7	0.37	600.43	91542.6	0.07	-1.70	91542.6	17.44	-1.70
75Affinity-1	73712.8	0.11	600.19	74113.5	0.07	-0.54	74113.5	22.45	-0.54
100Chain-1	136193.7	0.00	0.52	138164.7	0.06	-1.45	138164.7	41.04	-1.45
100Graph-1	143931.9	0.35	600.59	146179.2	0.10	-1.56	146179.2	50.18	-1.56
100Affinity-1	131256.2	0.26	600.18	132139.6	0.21	-0.67	132139.6	25.21	-0.67
25Chain-2	13053.2	0.00	0.03	13069.2	0.02	-0.12	13069.2	0.34	-0.12
25Graph-2	17130.7	0.00	0.04	17150.8	0.02	-0.12	17150.8	0.22	-0.12
25Affinity-2	12972.7	0.00	0.03	13100.4	0.02	-0.98	13100.4	0.34	-0.98
50Chain-2	45455.0	0.01	0.55	45755.2	0.03	-0.66	45755.2	1.49	-0.66
50Graph-2	38012.7	0.00	0.08	38245.9	0.03	-0.61	38245.9	4.84	-0.61
50Affinity-2	44624.8	0.01	39.74	46671.2	0.07	-4.58	46671.2	6.17	-4.58
75Chain-2	76162.1	1.14	600.51	82229.7	0.06	-7.97	82229.7	16.47	-7.97
75Graph-2	70060.1	0.33	600.44	72021.8	0.07	-2.80	72021.8	8.71	-2.80
75Affinity-2	81448.4	0.14	600.18	82466.4	0.09	-1.25	82466.4	5.10	-1.25
100Chain-2	129422.3	0.44	600.64	131598.4	0.13	-1.68	131598.4	34.10	-1.68
100Graph-2	130090.4	0.40	600.53	131458.9	0.15	-1.05	131458.9	37.08	-1.05
100Affinity-2	131498.8	0.32	600.21	132373.5	0.13	-0.66	132373.5	37.22	-0.66
25Chain-3	2721.5	0.01	1.01	2712.2	0.05	0.34	2712.2	0.69	0.34
25Graph-3	1832.6	0.01	1.06	1876.0	0.07	-2.37	1876.0	0.44	-2.37
25Affinity-3	1902.1	0.01	0.80	1877.9	0.06	1.27	1902.1	0.37	0.00
50Chain-3	5637.0	5.44	600.26	5796.6	0.15	-2.83	5796.6	1.48	-2.83
50Graph-3	5591.4	1.57	600.42	5572.3	0.15	0.34	5572.3	1.59	0.34
50Affinity-3	4915.2	0.90	600.12	4905.8	0.17	0.19	4905.8	1.80	0.19
75Chain-3	0.0	100.00	600.72	11827.4	0.31	-inf	11827.4	12.20	-inf
75Graph-3	10607.0	1.85	600.89	10678.8	0.35	-0.68	10678.8	2.06	-0.68
75Affinity-3	0.0	100.00	600.12	13599.1	0.48	-inf	13599.1	1.54	-inf
100Chain-3	0.0	100.00	0.39	18539.3	0.63	-inf	18539.3	12.98	-inf
100Graph-3	0.0	100.00	0.39	18605.1	0.68	-inf	18605.1	2.80	-inf
100Affinity-3	0.0	100.00	0.42	23923.1	0.73	-inf	23923.1	4.14	-inf
25Chain-4	18365.1	0.00	0.07	18361.7	0.07	-1.47	18361.7	0.37	-1.47
25Graph-4	20614.0	0.00	0.13	20916.7	0.07	-1.47	20916.7	0.37	-1.47
25Affinity-4	20131.9	0.00	0.27	19607.8	0.08	2.60	19607.8	0.33	2.60
50Chain-4	57368.1	2.07	600.34	60460.7	0.22	-5.39	60460.7	3.56	-5.39

6.4. VARIAZIONE DEI PARAMETRI

Table 6.7: sprint capacity reduction results (capacity - 15%)

Istanza	CPLEX			Heuristic			Lagrangian Heuristic		
	Zopt	Gap	Time	Zheu	Time	GHeu	Zheu	Time	GHeu
paytv -2	90203.0	0.00	0.74	93089.0	0.05	-3.20	93089.0	19.05	-3.20
web -2	31717.5	0.00	0.24	31365.0	0.02	1.11	31553.5	112.80	0.52
25Chain -1	16151.8	0.00	0.10	16244.1	0.06	-0.57	16244.1	5.65	-0.57
25Graph -1	13076.4	0.00	0.14	13097.6	0.05	-0.16	13097.6	1.00	-0.16
25Affinity -1	17474.6	0.01	4.76	17401.7	0.09	0.42	17474.6	0.91	0.00
50Chain -1	40393.6	0.00	0.38	40945.3	0.09	-1.36	40945.3	10.35	-1.36
50Graph -1	33686.1	0.00	0.04	33866.2	0.02	-0.53	33866.2	4.96	-0.53
50Affinity -1	38875.5	0.01	57.26	38897.0	0.11	-0.05	38897.0	9.67	-0.05
75Chain -1	81841.0	4.02	600.63	88888.2	0.20	-8.61	88888.2	142.45	-8.61
75Graph -1	88315.3	0.41	600.34	89732.7	0.09	-1.60	89732.7	11.18	-1.60
75Affinity -1	72325.4	0.18	600.20	72918.7	0.10	-0.82	72918.7	18.79	-0.82
100Chain -1	0.0	100.00	601.03	135632.1	0.24	-inf	135632.1	23.64	-inf
100Graph -1	140559.4	0.49	600.60	142750.5	0.08	-1.56	142750.5	7.61	-1.56
100Affinity -1	0.0	100.00	600.21	129323.7	0.25	-inf	129323.7	7.78	-inf
25Chain -2	12839.2	0.00	0.04	12950.6	0.02	-0.87	12950.6	0.54	-0.87
25Graph -2	16735.7	0.00	0.04	16813.0	0.02	-0.46	16813.0	0.35	-0.46
25Affinity -2	12909.6	0.00	0.03	12972.6	0.02	-0.49	12972.6	0.46	-0.49
50Chain -2	44583.7	0.01	352.20	45109.9	0.04	-1.18	45109.9	2.66	-1.18
50Graph -2	36304.1	0.00	0.21	37346.2	0.07	-2.87	37346.2	5.02	-2.87
50Affinity -2	43753.9	0.01	88.63	45561.4	0.06	-4.13	45561.4	5.71	-4.13
75Chain -2	74033.3	1.98	600.49	80730.9	0.14	-9.05	80730.9	19.14	-9.05
75Graph -2	69434.8	0.00	0.34	70659.9	0.13	-1.76	70659.9	7.46	-1.76
75Affinity -2	79740.6	0.27	600.18	80675.3	0.17	-1.17	80675.3	9.97	-1.17
100Chain -2	0.0	100.00	600.94	128940.2	0.36	-inf	128940.2	9.78	-inf
100Graph -2	0.0	100.00	600.88	128919.9	0.37	-inf	128919.9	5.06	-inf
100Affinity -2	0.0	100.00	600.19	129787.6	0.41	-inf	129787.6	5.02	-inf
25Chain -3	0.0	100.00	0.04	2608.0	0.07	-inf	2608.0	1.42	-inf
25Graph -3	1781.0	0.00	1.06	1811.2	0.08	-1.69	1811.2	0.42	-1.69
25Affinity -3	0.0	100.00	0.05	1819.3	0.07	-inf	1819.3	0.58	-inf
50Chain -3	0.0	100.00	0.15	5656.9	0.20	-inf	5656.9	2.71	-inf
50Graph -3	0.0	100.00	600.36	5344.5	0.20	-inf	5344.5	1.06	-inf
50Affinity -3	0.0	100.00	0.12	4709.1	0.22	-inf	4709.1	1.41	-inf
75Chain -3	0.0	100.00	0.22	11295.5	0.37	-inf	11295.5	9.88	-inf
75Graph -3	0.0	100.00	0.20	10312.4	0.33	-inf	10312.4	2.99	-inf
75Affinity -3	0.0	100.00	0.25	12970.2	0.42	-inf	12970.2	2.28	-inf
100Chain -3	0.0	100.00	0.38	17632.9	0.67	-inf	17632.9	11.32	-inf
100Graph -3	0.0	100.00	0.39	17634.1	0.61	-inf	17634.1	3.19	-inf
100Affinity -3	0.0	100.00	0.42	22932.6	0.70	-inf	22932.6	5.58	-inf
25Chain -4	18278.3	0.00	0.06	18351.9	0.07	-0.40	18351.9	0.71	-0.40
25Graph -4	19516.4	0.00	0.47	20083.4	0.07	-2.90	20083.4	0.30	-2.90
25Affinity -4	19331.6	0.01	0.27	18867.3	0.08	2.40	18867.3	0.43	2.40
50Chain -4	0.0	100.00	600.39	58512.5	0.15	-inf	58512.5	1.74	-inf
50Graph -4	0.0	100.00	600.39	63455.7	0.20	-inf	63455.7	1.12	-inf

6.4.3 metodo aggiornato

Table 6.8: sprint reduction results (nSprint - 2)

Istanza	CPLEX			Heuristic			Improved Heuristic			Lagrangian Heuristic		
	Zopt	Gap	Time	Zheu	Time	GHeu	Zheu	Time	GHeu	Zheu	Time	GHeu
paytv-2	97746.0	0.00	0.08	98873.0	0.03	-1.15	98228.6	0.07	-0.49	98873.0	15.76	-1.15
web-2	32745.5	0.00	0.11	32450.9	0.01	0.90	31672.5	0.02	3.28	32683.6	45.44	0.19
25Chain-1	17000.6	0.00	0.01	17075.0	0.02	-0.44	17075.0	0.03	-0.44	17075.0	0.22	-0.44
25Graph-1	13515.1	0.01	0.10	13583.7	0.01	-0.51	13584.0	0.03	-0.51	13583.7	0.10	-0.51
25Affinity-1	18567.3	0.00	0.26	18298.0	0.01	1.45	16641.3	0.03	10.37	18505.1	0.34	0.34
50Chain-1	42449.3	0.00	0.03	42689.1	0.02	-0.56	42759.8	0.06	-0.73	42689.1	2.14	-0.56
50Graph-1	34802.7	0.00	0.04	35346.8	0.02	-1.56	35414.8	0.04	-1.76	35346.8	0.73	-1.56
50Affinity-1	40545.1	0.01	0.97	40623.2	0.03	-0.19	39142.3	0.06	3.46	40623.2	2.78	-0.19
75Chain-1	91864.8	0.00	0.17	93223.5	0.04	-1.48	93329.7	0.08	-1.59	93223.5	29.00	-1.48
75Graph-1	93196.4	0.01	1.29	94302.5	0.03	-1.19	94155.4	0.09	-1.03	94302.5	29.88	-1.19
75Affinity-1	76076.2	0.01	62.44	76525.3	0.05	-0.59	74865.3	0.10	1.59	76525.3	15.90	-0.59
100Chain-1	137043.9	0.01	153.73	142276.9	0.04	-3.82	142529.0	0.13	-4.00	142276.9	39.44	-3.82
100Graph-1	149527.6	0.22	600.55	151302.6	0.06	-1.19	151559.1	0.19	-1.36	151302.6	60.06	-1.19
100Affinity-1	136146.9	0.14	600.17	136426.7	0.12	-0.20	133898.8	0.16	1.65	136426.7	71.18	-0.20
25Chain-2	13384.9	0.00	0.02	13346.6	0.01	0.29	13346.6	0.03	0.29	13346.6	0.27	0.29
25Graph-2	17574.2	0.00	0.01	17596.1	0.01	-0.12	17596.1	0.03	-0.12	17596.1	0.15	-0.12
25Affinity-2	13371.9	0.00	0.01	13445.1	0.01	-0.55	12595.7	0.03	5.81	13445.1	0.28	-0.55
50Chain-2	46761.2	0.01	0.14	46986.5	0.02	-0.48	47019.7	0.06	-0.55	46986.5	1.42	-0.48
50Graph-2	38844.8	0.00	0.08	39487.7	0.02	-1.65	39488.8	0.07	-1.66	39487.7	3.63	-1.65
50Affinity-2	46153.1	0.01	10.44	47976.3	0.02	-3.95	44983.1	0.05	2.54	47976.3	2.90	-3.95
75Chain-2	83638.0	0.00	0.18	84754.3	0.03	-1.33	84769.8	0.09	-1.35	84754.3	16.39	-1.33
75Graph-2	74014.5	0.00	0.06	74264.7	0.04	-0.34	74329.2	0.11	-0.42	74264.7	19.47	-0.34
75Affinity-2	84371.2	0.01	590.22	85075.3	0.04	-0.83	83209.2	0.11	1.38	85075.3	28.35	-0.83
100Chain-2	134438.0	0.29	600.62	136437.4	0.08	-1.49	136523.2	0.15	-1.55	136437.4	27.02	-1.49
100Graph-2	134978.3	0.30	600.49	136397.7	0.07	-1.05	136523.2	0.15	-1.14	136397.7	59.64	-1.05
100Affinity-2	136477.0	0.21	600.15	137159.8	0.07	-0.50	131805.7	0.21	3.42	137159.8	46.57	-0.50
25Chain-3	2855.2	0.01	2.69	2864.0	0.04	-0.31	2826.5	0.08	1.01	2864.0	0.25	-0.31
25Graph-3	1935.7	0.01	15.60	1963.0	0.04	-1.41	1969.2	0.07	-1.73	1963.0	0.18	-1.41
25Affinity-3	2002.4	0.01	1.80	1966.5	0.04	1.79	1858.2	0.08	7.20	1994.9	0.35	0.37
50Chain-3	6262.1	0.82	600.23	6300.6	0.13	-0.61	6055.0	0.19	3.31	6300.6	2.68	-0.61
50Graph-3	5909.9	1.57	600.38	5952.4	0.12	-0.72	5920.7	0.15	-0.18	5952.4	2.04	-0.72
50Affinity-3	5229.6	0.66	600.29	5210.4	0.13	0.37	4995.3	0.18	4.48	5210.4	3.23	0.37
75Chain-3	12245.5	2.25	600.43	12672.7	0.20	-3.49	12460.8	0.30	-1.76	12672.7	24.64	-3.49
75Graph-3	11395.1	0.89	600.96	11467.9	0.36	-0.64	11250.3	0.43	1.27	11467.9	3.60	-0.64
75Affinity-3	14477.1	1.43	600.09	14323.2	0.47	1.06	13989.5	0.62	3.37	14436.8	5.42	0.28
100Chain-3	19520.5	1.93	600.61	19872.7	0.34	-1.80	19587.0	0.52	-0.34	19872.7	35.70	-1.80
100Graph-3	19952.5	0.80	600.66	19834.5	0.49	0.59	19391.4	0.59	2.81	19925.1	43.35	0.14
100Affinity-3	25457.7	2.64	600.31	25737.6	0.49	-1.10	24942.8	0.56	2.02	25737.6	23.64	-1.10
25Chain-4	19033.8	0.00	0.04	19183.5	0.03	-0.79	19190.7	0.06	-0.82	19183.5	0.60	-0.79
25Graph-4	21261.5	0.00	0.16	21784.2	0.04	-2.46	21802.2	0.07	-2.54	21784.2	0.25	-2.46
25Affinity-4	21216.1	0.00	0.15	20638.5	0.04	2.72	19842.9	0.09	6.47	20638.5	0.17	2.72
50Chain-4	61557.3	0.01	0.98	63368.7	0.09	-2.94	63362.2	0.14	-2.93	63368.7	4.05	-2.94
50Graph-4	66307.3	0.78	600.37	67815.8	0.10	-2.27	67836.8	0.14	-2.31	67815.8	3.58	-2.27
50Affinity-4	60073.0	0.01	527.66	60005.8	0.12	0.11	58195.6	0.18	3.13	60005.8	5.39	0.11
75Chain-4	111001.1	5.32	600.85	119950.6	0.25	-8.06	120103.4	0.33	-8.20	119950.6	34.09	-8.06
75Graph-4	130536.6	1.05	600.32	132829.3	0.29	-1.76	132521.4	0.39	-1.52	132829.3	11.27	-1.76
75Affinity-4	125032.9	0.95	600.51	126628.4	0.32	-1.52	123784.7	0.38	1.00	126928.4	13.10	-1.52
100Chain-4	0.0	100.00	600.70	231268.0	0.36	-inf	230929.5	0.46	-inf	231268.0	150.92	-inf
100Graph-4	247404.2	0.66	600.40	249157.3	0.38	-0.71	249181.5	0.49	-0.72	249157.3	25.73	-0.71
100Affinity-4	237768.5	0.83	600.13	238076.4	0.46	-0.13	236102.5	0.57	0.70	238076.4	25.14	-0.13

6.5 Confronto funzioni obbiettivo

In questa sezione vengono confrontati i risultati ottenuti utilizzando le due diverse funzioni obbiettivo descritte nella sezione 5.4. Si userà come riferimento la tabella 6.9 la quale riporta i risultati ottenuti con la funzione obbiettivo (4.1) e la tabella 6.10 che riporta i risultati delle metriche calcolate su tali risultati.

Table 6.9: results con funzione obbiettivo (4.1)

CPLEX	Heuristic			Lagrangian Heuristic					
Istanza	Zopt	Gap	Time	Zheu	Time	GHeu	Zheu	Time	GHeu
instance_06_01	44354.6	0.01	0.23	42288.7	0.03	4.66	42288.7	0.23	4.66
instance_06_02	43181.6	0.01	0.42	41593.8	0.02	3.68	41593.8	0.22	3.68
instance_06_03	41367.7	0.01	0.30	40156.1	0.02	2.93	40156.1	0.32	2.93
instance_06_04	34073.1	0.01	0.11	27926.6	0.03	18.04	27926.6	0.33	18.04
instance_06_05	34991.9	0.00	0.38	31021.3	0.03	11.35	31021.3	0.16	11.35
instance_20_01	182020.7	1.02	300.10	178070.4	0.10	2.17	178070.4	3.01	2.17
instance_20_02	203651.0	0.29	300.07	190858.2	0.12	6.28	190858.2	4.00	6.28
instance_20_03	193769.0	0.01	217.66	177088.2	0.14	8.61	177088.2	3.03	8.61
instance_20_04	173408.3	0.01	291.01	172678.6	0.14	0.42	172678.6	4.00	0.42
instance_20_05	191724.4	3.38	300.05	170264.4	0.21	11.19	170264.4	2.46	11.19
instance_40_01	821010.9	0.05	300.12	804849.4	0.18	1.97	804849.4	18.34	1.97
instance_40_02	859090.7	0.08	300.12	826360.9	0.21	3.81	826360.9	14.45	3.81
instance_40_03	822323.5	0.10	300.11	801463.7	0.15	2.54	801463.7	22.44	2.54
instance_40_04	788923.6	0.48	300.13	773077.8	0.26	2.01	773077.8	27.66	2.01
instance_40_05	789186.1	0.26	300.12	794218.8	0.20	-0.64	794218.8	23.21	-0.64
instance_100_01	5492730.3	3.91	301.25	5638787.1	1.78	-2.66	5638787.1	300.18	-2.66
instance_100_02	5361497.2	6.61	300.93	5647903.4	1.75	-5.34	5647903.4	300.13	-5.34
instance_100_03	5613547.8	1.82	301.11	5631867.9	1.60	-0.33	5631867.9	300.72	-0.33
instance_100_04	5725382.8	5.00	300.91	5890661.9	1.84	-2.89	5890661.9	300.26	-2.89
instance_100_05	5430805.7	2.16	301.12	5530004.2	1.85	-1.83	5530004.2	300.02	-1.83

6.5. CONFRONTO FUNZIONI OBIETTIVO

Table 6.10: KPI con funzione obiettivo (4.1)

Istanza	CPLEX			Heuristic			Lagrangian Heuristic		
	N.U.S.	A.S.U.	H.U.S.	N.U.S.	A.S.U.	H.U.S.	N.U.S.	A.S.U.	H.U.S.
instance_06_01	5	95.59%	2	5	95.38%	2	5	95.38%	2
instance_06_02	7	84.68%	2	7	84.20%	2	7	84.20%	2
instance_06_03	6	88.89%	2	6	90.71%	2	6	90.71%	2
instance_06_04	5	86.65%	2	5	86.10%	2	5	86.10%	2
instance_06_05	5	98.34%	2	6	81.43%	2	6	81.43%	2
instance_20_01	23	95.94%	9	24	91.64%	8	24	91.64%	8
instance_20_02	21	93.09%	7	21	92.89%	6	21	92.89%	6
instance_20_03	21	95.87%	8	22	90.88%	7	22	90.88%	7
instance_20_04	23	96.12%	9	24	91.59%	8	24	91.59%	8
instance_20_05	21	95.91%	7	21	96.09%	7	21	96.09%	7
instance_40_01	21	96.16%	6	21	96.24%	6	21	96.24%	6
instance_40_02	19	95.29%	5	19	95.27%	5	19	95.27%	5
instance_40_03	20	99.20%	7	21	94.30%	5	21	94.30%	5
instance_40_04	22	97.72%	7	22	97.67%	6	22	97.67%	6
instance_40_05	23	95.06%	6	23	95.06%	6	23	95.06%	6
instance_100_01	103	91.35%	32	95	98.74%	29	95	98.74%	29
instance_100_02	94	91.49%	31	88	97.58%	29	88	97.58%	29
instance_100_03	92	92.68%	29	87	98.08%	27	87	98.08%	27
instance_100_04	101	90.10%	32	92	98.62%	28	92	98.62%	28
instance_100_05	96	92.06%	30	90	98.73%	27	90	98.73%	27
instance_f_01	9	92.56%	2	9	92.10%	2	9	92.10%	2
instance_f_02	10	93.61%	3	10	93.12%	2	10	93.12%	2
instance_f_03	9	95.56%	2	9	95.57%	2	9	95.57%	2
instance_f_04	10	93.14%	2	10	93.17%	2	10	93.17%	2
instance_f_05	9	95.09%	3	9	94.16%	2	9	94.16%	2

6.5. CONFRONTO FUNZIONI OBBIETTIVO

Table 6.11: KPI con funzione obbiettivo (4.1)

Istanza	CPLEX						Heuristic						Lagrangian Heuristic					
	D.R.	R.MAX	R.MIN	D.U.	U.MAX	U.MIN	D.R.	R.MAX	R.MIN	D.U.	U.MAX	U.MIN	D.R.	R.MAX	R.MIN	D.U.	U.MAX	U.MIN
instance_06_01	2.2	10.6	5.1	2.1	10.1	4.3	2.5	11.3	4.7	2.0	10.4	4.9	2.5	11.3	4.7	2.0	10.4	4.9
instance_06_02	2.3	9.2	1.8	2.0	7.7	1.6	2.9	11.7	1.8	2.5	10.3	1.6	2.9	11.7	1.8	2.5	10.3	1.6
instance_06_03	3.6	13.5	1.9	4.3	15.7	1.7	4.3	15.1	2.3	4.5	16.4	3.5	4.3	15.1	2.3	4.5	16.4	3.5
instance_06_04	3.1	11.2	3.0	3.0	11.1	3.4	4.2	12.4	2.6	4.2	13.2	2.7	4.2	12.4	2.6	4.2	13.2	2.7
instance_06_05	2.0	10.1	4.0	2.1	9.8	3.8	3.3	11.8	1.1	2.9	11.0	1.1	3.3	11.8	1.1	2.9	11.0	1.1
instance_20_01	1.1	6.4	2.5	0.9	5.9	2.7	1.3	6.6	1.1	1.1	5.9	1.4	1.3	6.6	1.1	1.1	5.9	1.4
instance_20_02	1.5	7.6	1.6	1.1	6.1	1.2	1.4	7.2	1.4	1.1	6.5	2.0	1.4	7.2	1.4	1.1	6.5	2.0
instance_20_03	1.5	6.5	1.2	1.1	5.8	1.7	1.9	9.0	1.0	1.4	6.5	1.7	1.9	9.0	1.0	1.4	6.5	1.7
instance_20_04	1.3	7.8	2.4	1.1	7.5	2.5	1.5	7.5	1.1	1.1	6.6	1.7	1.5	7.5	1.1	1.1	6.6	1.7
instance_20_05	1.3	6.8	1.8	1.2	6.4	1.9	1.4	7.8	2.4	1.1	6.6	2.2	1.4	7.8	2.4	1.1	6.6	2.2
instance_40_01	4.2	20.0	2.7	3.0	15.5	3.3	4.4	21.8	2.7	3.3	17.9	3.4	4.4	21.8	2.7	3.3	17.9	3.4
instance_40_02	4.5	19.9	1.7	4.2	19.2	1.5	5.5	28.2	1.4	5.0	25.5	1.6	5.5	28.2	1.4	5.0	25.5	1.6
instance_40_03	4.1	23.6	4.7	3.2	19.8	5.1	4.8	22.0	1.5	4.0	19.9	1.6	4.8	22.0	1.5	4.0	19.9	1.6
instance_40_04	3.1	19.0	4.7	2.8	18.4	5.2	3.5	19.7	3.3	2.8	17.5	4.5	3.5	19.7	3.3	2.8	17.5	4.5
instance_40_05	4.8	24.4	1.3	4.2	21.7	1.5	4.9	24.5	1.0	3.9	19.4	2.0	4.9	24.5	1.0	3.9	19.4	2.0
instance_100_01	1.6	9.9	1.4	1.1	7.4	1.7	1.5	9.2	2.2	1.0	7.1	2.7	1.5	9.2	2.2	1.0	7.1	2.7
instance_100_02	1.6	9.8	1.2	1.3	8.1	1.1	1.6	10.9	1.1	1.3	8.2	1.8	1.6	10.9	1.1	1.3	8.2	1.8
instance_100_03	1.7	10.1	1.1	1.3	7.9	1.4	1.7	10.8	1.1	1.1	7.3	1.8	1.7	10.8	1.1	1.1	7.3	1.8
instance_100_04	1.7	9.4	1.1	1.2	7.4	1.8	1.7	10.4	2.1	1.1	7.7	2.7	1.7	10.4	2.1	1.1	7.7	2.7
instance_100_05	1.5	8.6	1.3	1.3	7.6	1.1	1.6	9.0	2.3	1.2	8.2	2.8	1.6	9.0	2.3	1.2	8.2	2.8
instance_f01	8.6	26.7	1.5	8.2	25.7	1.7	10.8	36.6	1.5	10.8	36.7	1.7	10.8	36.6	1.5	10.8	36.7	1.7
instance_f02	5.6	19.7	2.4	4.6	17.2	2.8	7.6	30.0	3.2	6.7	26.3	2.9	7.6	30.0	3.2	6.7	26.3	2.9
instance_f03	9.8	35.2	2.3	8.4	30.7	3.2	9.4	34.0	2.3	8.2	30.5	3.4	9.4	34.0	2.3	8.2	30.5	3.4
instance_f04	8.7	33.2	2.3	8.2	31.8	3.3	9.8	36.4	2.5	9.3	35.3	2.5	9.8	36.4	2.5	9.3	35.3	2.5
instance_f05	6.0	21.8	2.7	5.5	20.7	3.2	8.6	29.1	2.7	8.1	27.2	3.0	8.6	29.1	2.7	8.1	27.2	3.0

come si notare nella tabella 6.9 nella colonna del tempo di esecuzione relativa alla esecuzione di CPLEX, già dal secondo gruppo di istanze il tempo di esecuzione e sempre al limite massimo di 300 secondi, questo indica che CPLEX non riesce a trovare la soluzione ottima in tempi ragionevoli.

6.5.1 funzune obbiettivo (5.23)

Nella tabella 6.12 vi e' riportato il confronto tra la funzione obbiettivo (4.1) e la funzione obbiettivo (5.23) in termini di tempo di esecuzione. come si pou notare vi differenze tra i tempi di esecuzione delle 2 funzioni obbiettivo, non considerando CPLEX che come già detto tende a saturare il tempo massimo di esecuzione, i tempi di esecuzione del algoritmo euristico con la funzione obbiettivo (5.23) sono il 16.3% piu veloci rispetto quelli della funzione obbiettivo (4.1), mentre i tempi di esecuzione dell'euristica lagrangiana variano soli di un 2.2% sempre a favore della funzione obbiettivo(5.23). Per quanto riguarda i KPI non sono presenti differenze , questo e' dovuto al fatto che entrambe le funzioni obbiettivo mirano a minimizzare il numero di slot utilizzati e la varianza del rischio, pertanto le soluzioni ottenute sono simili in termini di qualita' e l'inserimento dei parametri dp_{ij} e ub_{ij} ha portato semplicemente a un miglioramento dei tempi di esecuzione delle euristiche senza

influire sulla soluzione.

Table 6.12: confronto tempi di esecuzione tra le due funzioni obbiettivo (4.1) e (5.23)

Istanza	Funzioni obbiettivo(4.1)				Funzioni obbiettivo(5.23)			
	CPLEX Time	Heuristic Time	Lagrangian	Heuristic Time	CPLEX Time	Heuristic Time	Lagrangian	Heuristic Time
instance_06_01	0.23	0.03		0.23	0.13	0.02		0.16
instance_06_02	0.42	0.02		0.22	0.43	0.02		0.38
instance_06_03	0.30	0.02		0.32	0.26	0.02		0.28
instance_06_04	0.11	0.03		0.33	0.06	0.03		0.22
instance_06_05	0.38	0.03		0.16	0.30	0.02		0.14
instance_20_01	300.10	0.10		3.01	300.09	0.09		2.10
instance_20_02	300.07	0.12		4.00	300.11	0.10		3.34
instance_20_03	217.66	0.14		3.03	200.22	0.13		2.40
instance_20_04	291.01	0.14		4.00	88.79	0.12		2.63
instance_20_05	300.05	0.21		2.46	300.12	0.13		2.82
instance_40_01	300.12	0.18		18.34	300.10	0.15		20.06
instance_40_02	300.12	0.21		14.45	300.10	0.18		15.56
instance_40_03	300.11	0.15		22.44	300.10	0.23		21.00
instance_40_04	300.13	0.26		27.66	300.12	0.16		26.84
instance_40_05	300.12	0.20		23.21	300.14	0.14		23.19
instance_100_01	301.25	1.78		300.18	301.18	1.68		300.44
instance_100_02	300.93	1.75		300.13	300.92	1.48		300.73
instance_100_03	301.11	1.60		300.72	301.07	1.34		300.73
instance_100_04	300.91	1.84		300.26	302.31	1.65		300.26
instance_100_05	301.12	1.85		300.02	301.10	1.53		300.34

6.5.2 Funzione obbiettivo (5.24) e (5.25)

I risultati ottenuti utilizzando le funzioni obbiettivo (5.24) e (5.25) come riportato nella tabella 6.13, che indica i tempi di esecuzione delle 2 funzioni obbiettivo, come nella relazione tra le funzioni obbiettivo (4.1) e (5.23), vi e' un discostamento a per quanto riguarda i tempi di esecuzione dell'algoritmo euristico in cui la funzione obbiettivo (5.25) risulta piu veloce del 15.1% rispetto alla sua controparte (5.24), mentre per quanto riguarda l'euristica lagrangiana i tempi di esecuzione non si discostano in modo significativo. Per quanto riguarda i KPI non vi sono presenti differenze significative tra le 2 funzioni obbiettivo.

6.5. CONFRONTO FUNZIONI OBBIETTIVO

Table 6.13: confronto tempi di esecuzione tra le due funzioni obbiettivo (5.24) e (5.25)

Istanza	Funzioni obbiettivo(5.24)			Funzioni obbiettivo(5.25)		
	CPLEX Time	Heuristic Time	Lagrangian Heuristic Time	CPLEX Time	Heuristic Time	Lagrangian Heuristic Time
instance_06_01	0.12	0.02	0.14	0.03	0.02	0.16
instance_06_02	0.39	0.02	0.24	0.26	0.02	0.34
instance_06_03	0.36	0.02	0.41	0.46	0.03	0.41
instance_06_04	0.09	0.04	0.27	0.03	0.02	0.27
instance_06_05	0.24	0.02	0.15	0.42	0.02	0.15
instance_20_01	300.09	0.09	2.11	300.07	0.09	2.52
instance_20_02	300.10	0.13	3.37	128.92	0.10	3.57
instance_20_03	300.10	0.15	2.39	271.07	0.13	2.37
instance_20_04	232.82	0.13	2.59	287.18	0.10	2.67
instance_20_05	300.11	0.15	2.27	300.05	0.12	2.00
instance_40_01	300.13	0.23	16.53	300.12	0.27	17.70
instance_40_02	300.12	0.15	14.26	300.10	0.16	16.10
instance_40_03	300.09	0.28	19.72	300.10	0.15	17.05
instance_40_04	300.11	0.21	24.39	300.16	0.15	23.99
instance_40_05	300.12	0.21	22.86	300.12	0.19	25.55
instance_100_01	301.24	1.72	300.47	301.20	1.61	300.01
instance_100_02	300.98	1.66	300.42	300.94	1.52	300.01
instance_100_03	301.12	1.43	300.71	301.10	1.46	300.60
instance_100_04	300.97	1.81	300.19	300.94	1.91	300.60
instance_100_05	301.10	1.64	300.52	301.10	1.59	300.27

Chapter 7

Conclusioni

Bibliography

- [AA03] Oğuzhan Alagöz and Meral Azizoglu. Rescheduling of identical parallel machines under machine eligibility constraints. *European Journal of Operational Research*, 149(3):523–532, 2003.
- [ABV10] P. Avella, M. Boccia, and I. Vasilyev. A computational study of exact knapsack separation for the generalized assignment problem. *Computational Optimization and Applications*, 45:543–555, 2010.
- [AWSR03a] P. Abrahamsson, J. Warsta, M.T. Siponen, and J. Ronkainen. New directions on agile methods: a comparative analysis. In *25th International Conference on Software Engineering, 2003. Proceedings.*, pages 244–254, 2003.
- [AWSR03b] Pekka Abrahamsson, Juhani Warsta, Mikko T. Siponen, and Jussi Ronkainen. New directions on agile methods: A comparative analysis. In *Proc. ICSE*, pages 244–254, 2003.
- [AZDCG18] Wisam Haitham Abboud Al-Zubaidi, Hoa Khanh Dam, Morakot Choetkiertikul, and Aditya Ghose. Multi-objective iteration planning in agile development. In *2018 25th Asia-Pacific Software Engineering Conference (APSEC)*, pages 484–493, 2018.
- [B⁺01] K. Beck et al. Manifesto for agile software development. <http://agilemanifesto.org>, 2001.
- [BBvB⁺01] Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith,

- Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, and Dave Thomas. Manifesto for agile software development, 2001.
- [BDM⁺99] Peter Brucker, Andreas Drexler, Rolf Möhring, Klaus Neumann, and Erwin Pesch. Resource-constrained project scheduling: Notation, classification, models, and methods. *European Journal of Operational Research*, 112(1):3–41, 1999.
- [BGRT14] Marco A. Boschetti, Matteo Golfarelli, Stefano Rizzi, and Elisa Turricchia. A lagrangian heuristic for sprint planning in agile software development. *Computers & Operations Research*, 43:116–128, 2014.
- [BHM02] M.A. Boschetti, E. Hadjinconstantinou, and A. Mingozzi. New upper bounds for the finite two-dimensional orthogonal non-guillotine cutting stock problem. *IMA Journal of Management Mathematics*, 13:95–119, 2002.
- [BM03] M.A. Boschetti and A. Mingozzi. The two-dimensional finite bin packing problem. Part I: New lower bounds for the oriented case. *JOR*, 1:27–42, 2003.
- [BM10] Marco Antonio Boschetti and Lorenza Montaletti. An exact algorithm for the two-dimensional strip-packing problem. *Operations Research*, 58(6):1774–1791, November 2010.
- [Coh04a] Mike Cohn. *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional, 2004.
- [Coh04b] Mike Cohn. *User stories applied: For agile software development*. Addison-Wesley Professional, 2004.
- [Coh05] Mike Cohn. *Agile estimating and planning*. Pearson Education, 2005.
- [DB98] Ronald De Boer. *Resource-constrained multi-project management*. PhD thesis, PhD thesis, University of Twente, The Netherlands, 1998.

- [DCH03] Mark Denne and Jane Cleland-Huang. *Software by numbers: Low-risk, high-return development*. Prentice Hall Professional, 2003.
- [DD08a] Tore Dybå and Torgeir Dingsøy. Empirical studies of agile software development: A systematic review. *Information & Software Technology*, 50(9-10):833–859, 2008.
- [DD08b] Tore Dybå and Torgeir Dingsøy. Empirical studies of agile software development: A systematic review. *Information and Software Technology*, 50(9):833–859, 2008.
- [DH02] Erik L Demeulemeester and Willy S Herroelen. *Project scheduling: a research handbook*. Springer, 2002.
- [GR04] D Greer and G Ruhe. Software release planning: an evolutionary and iterative approach. *Information and Software Technology*, 46(4):243–253, 2004.
- [GRT12] Matteo Golfarelli, Stefano Rizzi, and Elisa Turricchia. Sprint planning optimization in agile data warehouse design. pages 30–41, 09 2012.
- [GRT13] Matteo Golfarelli, Stefano Rizzi, and Elisa Turricchia. Multi-sprint planning and smooth replanning: An optimization model. *Journal of Systems and Software*, 86(9):2357–2370, 2013.
- [GS05] Noud Gademann and Marco Schutten. Linear-programming-based heuristics for project capacity planning. *IIE Transactions (Institute of Industrial Engineers)*, 37(2):153 – 165, 2005. Cited by: 31.
- [HDDR99] Willy Herroelen, Erik Demeulemeester, and Bert De Reyck. A classification scheme for project scheduling. In *Project scheduling: recent models, algorithms and applications*, pages 1–26. Springer, 1999.
- [HLD02] Willy Herroelen, Roel Leus, and Erik Demeulemeester. Critical chain project scheduling: Do not oversimplify. *Project Management Journal*, 33(4):48–60, 2002.

- [KGW13] Mahvish Khurum, Tony Gorschek, and Magnus Wilson. The software value map—an exhaustive collection of value aspects for the development of software intensive products. *Journal of software: Evolution and Process*, 25(7):711–741, 2013.
- [KP01] R Kolisch and R Padman. An integrated survey of deterministic project scheduling. *Omega*, 29(3):249–272, 2001.
- [kS11] Ákos Szöke. Conceptual scheduling model and optimized release scheduling for agile environments. *Information and Software Technology*, 53(6):574–591, 2011. Special Section: Best papers from the APSEC.
- [LvdABD10] Chen Li, Marjan van den Akker, Sjaak Brinkkemper, and Guido Diepen. An integrated approach for requirement selection and scheduling in software release planning. *Requirements Engineering*, 15(4):375 – 396, 2010. Cited by: 48; All Open Access, Hybrid Gold Open Access.
- [New98] Robert C Newbold. *Project management in the fast lane: applying the theory of constraints*. CRC Press, 1998.
- [PP04] Yongtae Park and Gwangman Park. A new method for technology valuation in monetary value: procedure and application. *Technovation*, 24(5):387–394, 2004.
- [PSW94] Adri Platje, Harald Seidel, and Sipke Wadman. Project and portfolio planning cycle: Project-based management for the multiproject challenge. *International Journal of Project Management*, 12(2):100–106, 1994.
- [RFB09] Mikko Rönkkö, Christian Frühwirth, and Stefan Biffl. Integrating value and utility concepts into a value decomposition model for value-based software engineering. In *International Conference on Product-Focused Software Process Improvement*, pages 362–374. Springer, 2009.

BIBLIOGRAPHY

- [Sch95] K. Schwaber. SCRUM development process. In *Proc. OOPSLA*, 1995.
- [Sch97] Ken Schwaber. Scrum development process. In Jeff Sutherland, Cory Casanave, Joaquin Miller, Philip Patel, and Glenn Hollowell, editors, *Business Object Design and Implementation*, pages 117–134, London, 1997. Springer London.
- [SOS93] Norman Sadeh, Shinichi Otsuka, and R Schedlback. Predictive and reactive scheduling with the microboss production scheduling and control system. In *Proceedings of the IJCAI-93 workshop on knowledge-based production planning, scheduling and control*, pages 293–306, 1993.
- [SR07] Moshood Omolade Saliu and Guenther Ruhe. Bi-objective release planning for evolving software systems. In *Proceedings of the the 6th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on The Foundations of Software Engineering*, ESEC-FSE '07, page 105–114, New York, NY, USA, 2007. Association for Computing Machinery.
- [SW00] Hani El Sakkout and Mark Wallace. Probe backtrack search for minimal perturbation in dynamic scheduling. *Constraints*, 5(4):359–388, 2000.
- [vTdP11] Gert van Valkenhoef, Tommi Tervonen, Bert de Brock, and Douwe Postmus. Quantitative release planning in extreme programming. *Information and Software Technology*, 53(11):1227–1235, 2011. AMOST 2010.

Acknowledgements

Optional. Max 1 page.